



Technical University of Denmark

DTU Compute

Automated Historical Cause-of-Death Coding Using Fine-Tuned Language Models

Bachelor's Thesis

June 2026

Lucas E. Bülow Mortensen

Elias Dahl Lunøe

Supervisors: Nicki Detlefsen, Tim Riswick, Mads Perner

Abstract

Historical mortality registers record each death as a free-text cause, written in the language and orthography of its time and place. Before such data can be analyzed, every entry must be mapped by hand to a standardized historical disease code (ICD10h), expert work slow enough to gate whole research projects. This thesis asks whether a fine-tuned, task-specific language model can take over the bulk of that coding within a human-in-the-loop workflow, and whether the confidence of its predictions can make it safe to deploy.

We fine-tune the FLAN-T5 sequence-to-sequence model at four sizes on roughly 1.5 million hand-coded records from five European registers spanning Danish, Dutch, English, French, and Spanish, and evaluate the resulting deployment model, `codLLM-base-deploy`, on a locked test split against classical, retrieval-based, and agentic baselines, under uncertainty-based selective prediction, and under transfer to an entirely held-out source. On the full test set it reaches 0.985 exact-match accuracy, 0.987 micro-F1, and 0.749 macro-F1, against 0.314 macro-F1 for the strongest training-free baseline, a TF-IDF classifier, and it far exceeds two frontier agents equipped with tools (0.56 to 0.60 accuracy on a matched subset) at a fraction of their per-record cost. On the memorization-free unseen-string slice, where the model is weakest, it still reaches 0.843 exact-match accuracy and 0.625 macro-F1, so its advantage reflects genuine generalization rather than recall of seen strings.

Reading the model’s own token-level confidence makes the workflow usable. Auto-accepting the most-confident 80% of predictions codes them at near-perfect accuracy (above 0.999) while routing the uncertain remainder to a historical demographer, and on the unseen slice the same signal lifts retained accuracy from 0.843 to 0.925, deferring the genuinely novel inputs rather than coding them confidently wrong. Transfer to a source the model never trained on is the hardest setting: exact-match accuracy on a held-out source ranges from 0.22 to 0.64 and macro-F1 from 0.09 to 0.36, and although confidence stays informative, no operating point reaches an accuracy that would license unsupervised coding, so on a genuinely new register the model is a decision-support tool rather than an autonomous coder. We treat the dataset’s heavy duplication explicitly throughout, reporting a memorization-free slice and duplication-aware confidence intervals, and recommend the compute-light base model for deployment.

Contents

Abstract	1
1 Introduction	5
2 Domain	7
2.1 The ICD10 classification system	7
2.2 ICD10h: a historical adaptation of ICD10	7
2.3 The masterlist	9
2.4 Manual coding in practice	9
3 Background	10
3.1 Models	10
3.2 Sequence-to-sequence task framing	11
3.3 Decoding and token-level uncertainty	12
3.4 Fine-tuning and optimization	13
3.5 Input noise and robustness	14
3.6 Evaluation metrics	14
4 Related Work	16
4.1 Automated coding of historical causes of death	16
4.2 Automated standardization of historical records	17
4.3 Automated clinical ICD coding	17
4.4 Selective prediction and uncertainty estimation	18
5 Data	19
5.1 Data contract and ethical framing	19
5.2 Sources	19
5.3 Descriptive statistics	21
5.3.1 Source conventions and historical wording	23
5.4 Splitting Strategy and Inherent Data Leakage	23
5.5 Masterlist and reference strings	25
6 Methodology	26
6.1 Experimental Setup	26
6.1.1 Runtime, Versioning, and Compute	26
6.1.2 Model and Fine-Tuning Contract	26
6.1.3 Data and Evaluation Protocol	28

6.1.4	Reproducibility	30
6.2	Methods	30
6.2.1	Pipeline and Output Contract	30
6.2.2	Controlled Comparisons and Experiment Grid	32
6.2.3	Baselines and Selective Prediction	34
6.2.4	Training Data Interventions	35
6.2.5	Discarded Alternatives	37
7	Experiments and Results	39
7.1	Model capacity and the deployment recipe	39
7.1.1	Coding specificity	40
7.1.2	Selective prediction	40
7.2	The deployment model	42
7.2.1	Qualitative error patterns	42
7.2.2	Subgroup biases	42
7.2.3	Comparison with baselines	43
7.3	Configuration ablations	44
7.4	Transfer to held-out sources	47
8	Discussion	49
8.1	Interpreting the results	49
8.2	Historical Data Shift and Robustness	51
8.3	Error Severity	55
8.4	Limitations	56
8.4.1	Dataset Quality	56
8.4.2	Limitations of the configuration search	56
8.4.3	Output validity	57
8.4.4	What the unseen-string slice measures	57
8.4.5	Statistical scope of the reported intervals	58
8.5	Societal Impact	58
8.6	Systematic Bias	59
9	Conclusion	61
10	Acknowledgments	64
	Appendix	66
A	Declaration of Generative AI Usage	66

B	Appendix	67
B.1	Data-use agreement	67
B.2	GitHub Repository	69
B.3	Compute environment snapshot	69
C	Agentic baseline prompt	70
D	Manual-coding effort: source correspondence	73
E	Per-chapter accuracy	75
F	Evaluation metric definitions	77
G	Specificity-level metrics	78
H	Held-out specificity-level metrics	79
	Bibliography	81

1 Introduction

How diseases spread through a population, who dies of them, and how circumstances of age, place, season, and social condition shape the risk of death are questions best studied across many communities and over long stretches of time. Much of the evidence at that scale comes from records of individual deaths, since parish registers, civil death registrations, and burial books document mortality both in ordinary years and through the epidemics that periodically swept past societies. Read together, these holdings let researchers trace how disease moved through past populations and how living conditions shaped who survived, the same evidence that informs how present-day public health understands disease and prepares for new outbreaks. Across Europe and beyond, archives of this material are now being digitized into structured, machine-readable data, millions of individual death records, each naming the deceased and recording an age, a place, a date, and a stated cause of death. The value of this growing body of evidence, however, depends on reading entries written in many places and periods in a common way.

To use such records beyond a single archive, deaths have to be counted the same way across countries, institutions, and centuries. That need is what the International Classification of Diseases provides. Now in its tenth revision (ICD10), it is a standardized, hierarchical vocabulary that maps free-text cause-of-death (CoD) statements onto fixed codes, giving the same code to the same underlying condition however it was phrased.

Historical sources make this harder. As parish registers, civil registries, and burial records are digitized, they produce free-text statements that predate the modern system and carry archaic disease names, regional terms, and diagnostic conventions ICD10 was never designed to hold. The historical-demography community answered this with ICD10h, an extension that keeps ICD10's structure but adds the codes needed to represent historical causes faithfully. Translating free text to ICD10h is still done by hand, by trained coders, and at scale it is slow and expensive. For a sizeable dataset the coding stage can run for months, often deciding whether a research question can be answered at all.

Automated ICD coding already exists for modern clinical text, but the historical setting is markedly harder. The same condition may appear in Danish, Dutch, English, Spanish, or French. Spelling shifts from scribe to scribe. Digitization adds character-level noise, whether from optical character recognition (OCR) of scanned registers or from keystroke slips in transcription. The label distribution is steeply long-tailed. Conventional methods struggle here, partly because they cannot easily exploit the linguistic regularities that connect, for example, an archaic Dutch tuberculosis term to its English counterpart. This thesis asks whether a fine-tuned, task-specific large language model (LLM) can take

over most ICD10h coding in a human-in-the-loop workflow, coding the confident majority itself and routing the uncertain minority to expert review. We fine-tune FLAN-T5 [1], a multilingually pretrained encoder-decoder model, on existing hand-coded European mortality datasets at four model sizes (small, base, large, xl) trained on identical splits, and refer to the resulting models as `codLLM-small`, `codLLM-base`, `codLLM-large`, and `codLLM-xl`. The one-factor experiments that inform the final recipe are all run at the base size. We call the resulting deployment model `codLLM-base-deploy`, the base-size model retrained with the settings they select. We pose three research questions:

RQ1. Does codLLM code historical causes of death more accurately than classical, retrieval-based, and agentic baselines? We compare against a TF-IDF linear classifier, a training-free multilingual sentence-embedding retriever, agentic LLM baselines (two frontier agents equipped with tools), and a majority-class floor on the full test set. Because `codLLM` is supervised on this task while the retrieval and agentic baselines are not, we also read `codLLM`’s own accuracy on the slice of strings never seen in training, the sharpest test of whether its advantage reflects genuine generalization rather than memorization. We expect `codLLM` to outperform all four baselines on the full test, and to keep a strong score on the unseen-string slice where memorization cannot inflate the result.

RQ2. Can uncertainty-informed selective prediction keep retained predictions highly accurate while deferring ambiguous cases to expert review? We test this on `codLLM-base-deploy`, ranking predictions by a per-record confidence taken from the model’s own token-level outputs and deferring the least confident. We then measure retained-set accuracy as the deferral rate rises, on both the full test and the unseen-string slice. We expect retained accuracy to rise as low-confidence records are deferred, giving a usable operating point where most records are auto-accepted at high accuracy and the ambiguous minority is routed to review.

RQ3. On a historical source the model never saw in training, does accuracy hold, and does the model’s confidence fall where it fails? In this worst-case scenario, we hold out each source in turn, train on the rest, and measure performance on the held-out source. A system safe to deploy should be unconfident exactly where it is wrong, so we test whether confidence drops, and deferral rises, on the sources where accuracy is worst. We expect accuracy to fall on a held-out source, but the model’s confidence to fall with it, so its errors are deferred rather than coded silently wrong.

The remainder of the thesis is organized as follows. Chapter 2 situates the work in its historical-demography context, Chapter 3 reviews the technical background, and Chapter 4 surveys prior automated approaches to ICD coding. Chapter 5 describes the datasets used, and Chapter 6 details the experimental setup and the methods used. Chapter 7 presents the results, Chapter 8 interprets them, and Chapter 9 concludes.

2 Domain

This chapter defines the domain concepts used in the thesis: ICD10, its historical adaptation ICD10h, the masterlist, and manual coding practice.

2.1 The ICD10 classification system

The International Classification of Diseases (ICD), maintained by the World Health Organization, is the standard international system for classifying diseases and causes of death. This thesis uses ICD10, specifically the 2016 version on which ICD10h is based. ICD10 assigns each cause to an alphanumeric code, making mortality data comparable across institutions, countries, and periods even when the original cause-of-death (CoD) statements differ in wording [2].

ICD10 is both a coding system and a classification system. At the highest level, it groups causes into 22 chapters, each covering a broad family of diseases or external causes. Chapter I, for example, covers infectious and parasitic diseases, while Chapter IX covers diseases of the circulatory system. Within each chapter, codes become progressively more specific, narrowing from the chapter range (A00–B99 for Chapter I) to three-character categories and then four-character subcategories. The code prefix therefore carries information about the position of a cause in the hierarchy.

For mortality coding, a further distinction matters. A death record may mention several conditions, but statistical coding usually seeks the underlying cause of death: the disease or injury that initiated the train of events leading directly to death. This differs from immediate modes of death, such as cardiac arrest, and from secondary conditions listed on the same record.

2.2 ICD10h: a historical adaptation of ICD10

ICD10 was designed for modern clinical and statistical use, not for historical death registers. Historical records contain archaic disease names, local terms, spelling variation, shifting diagnostic meanings, and vague expressions that do not always map cleanly onto modern ICD10 categories. ICD10h was created as a research tool for coding such individual-level historical causes of death while retaining a link to ICD10 [3]. It is based on ICD10 version 2016, but it is not an official WHO version or authorised extension of ICD10, and data coded in ICD10h are not directly equivalent to data coded under standard ICD10 rules [4].

One root-to-leaf path through the four ICD-10h levels

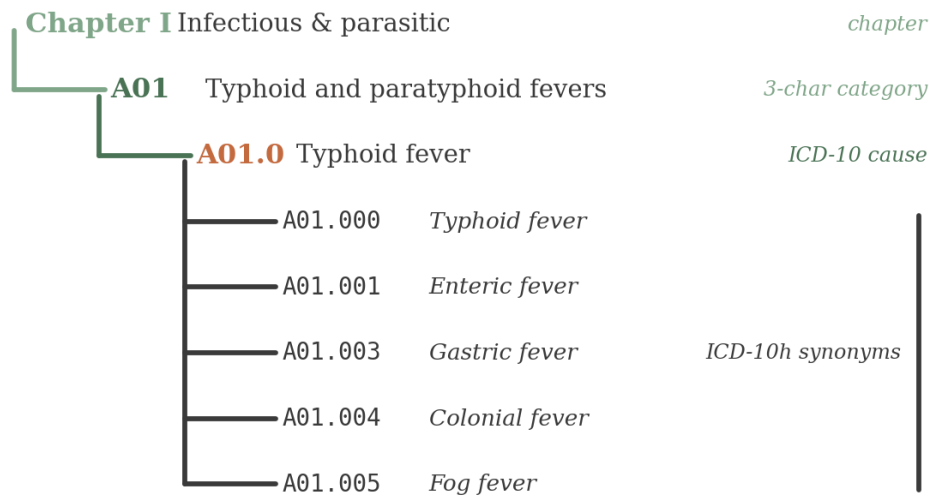


Figure 2.1: A single path through the ICD10h hierarchy. Chapter I contains category A01 (typhoid and paratyphoid fevers), which contains ICD10 subcategory A01.0 (typhoid fever). In ICD10h, this subcategory is represented by the padded code A01.000 and by additional historical-variant codes such as A01.001 enteric fever, A01.003 gastric fever, A01.004 colonial fever, and A01.005 fog fever.

ICD10h takes ICD10 codes as its starting point. Four-character ICD10 codes are converted by adding two zeros: A00.9, cholera unspecified, becomes A00.900. ICD10 codes without a decimal digit receive three zeros: A35, other tetanus, becomes A35.000. When a historical or archaic term has no exact ICD10 equivalent, ICD10h creates an additional code beneath the nearest appropriate ICD10 code. For example, apoplexy is coded as I64.001 under the ICD10 stroke category I64.000, and phthisis is coded under respiratory tuberculosis.

ICD10h is therefore not always ICD10 with extra suffixes appended. In most cases it preserves the ICD10 location of the corresponding modern condition, but it also makes small historically motivated departures from standard ICD10 coding logic. For example, ICD10h treats nineteenth-century cancers as fatal unless the record states otherwise, because historical death certificates rarely contain the laboratory confirmation assumed by modern ICD10. More generally, ICD10h prioritizes stable coding of historical diagnostic expressions over strict reproduction of every modern ICD10 mortality rule.

A central rule is that ICD10h codes the diagnostic word or expression, not the coder’s retrospective interpretation of what the condition may have meant in a particular context. This keeps the same written cause linked to the same code across datasets and periods. If the meaning of a term changes over time, or if a term should be grouped differently for a particular historical analysis, that adjustment belongs to the later categorization step rather than to the initial code assignment.

In this thesis, ICD10h is the target label space: the set of codes the model learns to

assign to each free-text cause of death.

2.3 The masterlist

The operational reference for ICD10h coding is the masterlist [4], a table with one row per ICD10h code. Each row records the ICD10h code, the ICD10 code it maps to, inherited ICD10 descriptions, the historical description that distinguishes the code from its siblings, a coarse historical-category grouping, and coding-rule flags. These flags mark constraints a human coder would otherwise have to remember: 2,396 codes are not valid for underlying cause-of-death, 257 are sex-specific, and 301 are deprecated. The masterlist’s role in the data pipeline is detailed in Chapter 5 (Section 5.5).

2.4 Manual coding in practice

Trained coders, typically historical demographers with domain knowledge of period medicine, assign codes by hand. A coder reads each free-text statement, which is often abbreviated, written in Latin or a regional vernacular, and shaped by the conventions of the registering institution, so a single condition can appear in many written forms. The coder interprets the statement in its historical and linguistic context and selects the matching ICD10h code from the masterlist. The coder must also resolve statements that name more than one condition by identifying the underlying cause, and must apply the coding rules that the masterlist flags encode. The full procedure coders follow, from reading a statement through resolving the underlying cause to selecting the ICD10h code, is set out in a published coding manual [4].

This work is slow and expensive because it requires scarce expertise and careful judgment on each record. The Copenhagen register used in this thesis makes the cost concrete. Its causes of death were hand-coded by Louise Perner over roughly 351 hours of expert work (L. Perner, recorded coding time, reported June 2026, see Appendix D), producing the Danish Historical Cause of Death Database, which records 10,551 distinct historical causes from 1861 to 1911 [5], an average of roughly two minutes per distinct cause. For the sizeable registers that historical demography increasingly produces through digitization, the coding stage can take months and becomes the step that determines whether a research question is feasible at all. Relieving this bottleneck is the practical motivation for the automated coder this thesis investigates.

The historical registers on which this coder is built and evaluated are characterized in detail in Chapter 5, and the modeling approach developed to meet them follows in Chapters 3 and 6.

3 Background

3.1 Models

This thesis uses the FLAN-T5 model family [1], which is based on T5, a Text-to-Text Transfer Transformer [6]. T5 frames natural-language tasks in a unified text-to-text format, where both the input and the desired output are represented as text. This means that tasks such as translation, summarization, question answering, and classification can all be expressed as conditional generation problems.

T5 uses an encoder-decoder Transformer architecture [7]. Both halves are built from self-attention layers, in which each position forms a weighted combination of the other positions, letting the model relate tokens across the whole sequence rather than only their neighbors. The encoder reads the input sequence and produces contextual representations. The decoder generates the output sequence one token at a time, attending both to its own prefix (causal self-attention) and to the encoder output (cross-attention), and at each step emits a probability distribution over the vocabulary through a softmax. Writing $z_t \in \mathbb{R}^{|V|}$ for the decoder logits at step t over the vocabulary V , the model assigns token v the probability

$$p_\theta(y_t = v \mid y_{<t}, x) = \frac{\exp(z_{t,v})}{\sum_{v' \in V} \exp(z_{t,v'})}.$$

That per-step distribution matters later. It is what the uncertainty signals of section 3.3 are read from. The architecture is well suited to tasks where the output is itself a structured text sequence rather than a single scalar prediction.

FLAN-T5 extends T5 through instruction fine-tuning. The model is further trained on a large collection of tasks phrased as instructions, which improves its zero-shot and few-shot generalization to unseen tasks [1]. The released FLAN-T5 checkpoints are documented for use across many languages, which makes them a plausible starting point for multilingual historical text. In this thesis, however, we do not rely on instruction-style prompting directly. We use FLAN-T5 as a pretrained encoder-decoder initialization and then fully fine-tune it for the specific ICD10h coding task described in the methodology. Its suitability for multilingual historical cause-of-death coding is therefore treated as an empirical question tested by the experiments, rather than assumed from the pretrained checkpoint alone.

The FLAN-T5 tokenizer uses SentencePiece subword tokenization [8]. Subword tokenization represents rare or unseen words as combinations of smaller units, rather than requiring every word to exist as a complete vocabulary entry. This is useful for noisy or historically variable text, where spelling variation and rare terms are common.

3.2 Sequence-to-sequence task framing

Cause-of-death coding is framed here as a sequence-to-sequence problem, where the model reads a free-text cause-of-death and *generates* the ICD10h code as text, one token at a time, rather than choosing from a fixed list of classes. ICD10h is a historically adapted extension of ICD10 whose codes are hierarchical, a chapter letter, a category, and trailing digits of increasing specificity (for example A16.200), as detailed in chapter 2. The input is the cause string, optionally prefixed with metadata fields, and the target is the canonical code string. A record with several causes becomes a short, separator-joined list of codes.

Framing the task this way rather than as classification over the roughly 14,000-way ICD10h label space with a softmax head has two advantages. First, it lets the decoder exploit the structure inside the code, since codes that share a chapter and category also share a prefix of output tokens, so the model can pick up the hierarchy implicitly through the tokens it emits, instead of treating every code as an unrelated class. Second, it extends to several causes per record for free. Emitting two or three codes is just a slightly longer target sequence, whereas a single-label classifier would have to be redesigned. The price is that outputs are not valid by construction, and the decoder can in principle emit a string that is not a real ICD10h code, which is why the exact-match and set-based metrics of section 3.6 are the right way to score it.

A practical wrinkle of generating codes as text is that the decoder does not emit one symbol per code but a sequence of SentencePiece subwords, and that sequence is uneven, since how a code splits depends on which digit runs and letters the tokenizer happens to carry as pieces. Table 3.1 shows a few examples under the FLAN-T5 tokenizer. The seven characters of A36.000 become four tokens. X19.000 becomes five, because the rarer chapter letter X is pushed onto a token of its own. A01.000 collapses to three, since the tokenizer happens to hold 1.000 as a single piece. The same mechanism fragments unfamiliar *input* far harder (the archaic Dutch *doodgeboorte*, stillbirth, breaks into eight pieces and a nonsense string into one piece per character), because SentencePiece prefers a segmentation assembled from many short, common pieces whenever it has no long learned subword for the string. That behavior is exactly what makes it robust to the spelling variation and out-of-vocabulary terms of historical text, at the cost of longer, more fragmented encodings for the rarest strings.

String	SentencePiece pieces	Tokens
A36.000	_A 36 . 000	4
C34.900	_C 34 . 900	4
A01.000	_A 0 1.000	3
X19.000	_ X 19 . 000	5
<i>doodgeboorte</i>	_do o d geb o or t e	8
<i>qwzrk</i>	_ q w z r k	6

Table 3.1: How the FLAN-T5 SentencePiece tokenizer splits a few ICD10h codes (top) and free strings (bottom), as produced by the `google/flan-t5` tokenizer. `_` marks the word-boundary token. The token count is content-dependent (the common chapter letter A merges with its boundary marker, the rare X does not).

This unevenness matters for how the model commits to a code. Because a code is emitted token by token, a frequent multi-digit run such as 36 or 900 is produced in a single confident step, while a rarer code is assembled from smaller pieces that spread the decision, and the model’s per-token uncertainty, across more steps. It also means the model can be *partly* right. It may emit a correct chapter letter and then a wrong trailing digit, landing on a real but neighboring code rather than on nonsense. Both effects feed the token-level confidence signals of section 3.3 and the hierarchy-level metrics of section 3.6.

3.3 Decoding and token-level uncertainty

Because the output is generated autoregressively, the model produces a code by repeatedly choosing the next token from its per-step vocabulary distribution. The simplest rule is greedy decoding, which takes the most probable token at each step. Beam search instead keeps several partial sequences and extends the most probable overall, trading compute for a lower chance of committing early to a poor prefix. The decoding rule fixes both the emitted code and the per-token probabilities from which confidence is read.

Those per-token probabilities give a calibration-free measure of how sure the model is. Let $p(y_t \mid y_{<t}, x)$ be the probability the decoder assigns to the token it emits at step t . Its log-probability $\log p(y_t \mid y_{<t}, x)$ is high when the model is confident and low when it hesitates, and the entropy of the full step distribution, $H_t = -\sum_v p_v \log p_v$ over the vocabulary, measures how spread the model’s choice was regardless of which token it finally picked. For a code generated over T steps, these per-token quantities aggregate into a single confidence score, for example the summed or mean log-probability, or the mean entropy,

$$s_{\text{sum}} = \sum_{t=1}^T \log p(y_t \mid y_{<t}, x), \quad s_{\text{mean}} = \frac{1}{T} \sum_{t=1}^T \log p(y_t \mid y_{<t}, x), \quad \bar{H} = \frac{1}{T} \sum_{t=1}^T H_t.$$

The summed score penalizes longer codes, as each step adds another negative term, while the mean does not, which interacts with the uneven token lengths of section 3.2. These are the signals used for selective prediction in subsection 6.2.3.

Selective prediction turns such a score s_i into a decision to answer or abstain at a threshold τ : the model returns a code when $s_i \geq \tau$ and defers the record to a human coder otherwise. Sweeping τ trades coverage, the fraction of records answered, against the exact-match accuracy retained on those answered records. Raising the threshold lowers coverage and should raise retained accuracy, so a useful operating point is the highest coverage at which retained accuracy stays above a target. This is the deployment question studied in subsection 7.1.2.

3.4 Fine-tuning and optimization

Fine-tuning adapts a model that has already been pretrained on a large general dataset to a specific task by continuing to train its weights on task-specific data. Because the pretrained weights already encode broad linguistic structure, fine-tuning typically needs far less data and far fewer updates than training from scratch [6]. It is the mechanism by which the general multilingual knowledge in FLAN-T5 is specialized to ICD10h coding. The model defines a left-to-right factorized distribution over output sequences, $p_\theta(y \mid x) = \prod_{t=1}^T p_\theta(y_t \mid y_{<t}, x)$, and fine-tuning minimizes the token-level negative log-likelihood of the true codes,

$$\mathcal{L}(\theta) = - \sum_{(x,y) \in \mathcal{D}} \sum_{t=1}^T \log p_\theta(y_t \mid y_{<t}, x).$$

The same $\log p_\theta$ supplies the per-token confidence signals of section 3.3.

The weights are updated by gradient descent with an optimizer from the Adam family [9], which adapts a per-parameter step size from running estimates of the gradient’s first and second moments. The overall step size is governed by a global *learning rate*, which is usually not held constant over training. A learning-rate *schedule* first *warms* the rate up from zero over an initial portion of training, avoiding large, destabilizing updates to the pretrained weights early on, and then *decays* it back towards zero so that later updates make progressively finer refinements. Two common decay shapes are *linear*, which lowers the rate at a constant pace, and *cosine* annealing [10], which keeps the rate higher through the early part of training, drops most steeply around the middle, and then flattens as it approaches zero. The initial warmup follows the gradual ramp used for stable large-batch training [11]. The optimizer, schedule, and concrete values used in this thesis are reported in subsection 6.1.2, and the choice of decay shape was made empirically rather than *a priori* (chapter 7).

3.5 Input noise and robustness

A model fit only on clean text can be brittle. A single changed character may flip a confident prediction, and historical sources supply exactly that kind of variation in abundance. A standard remedy is to perturb the inputs slightly during training. Training with input noise is, in expectation, a form of regularization. For a squared-error objective it is equivalent to a Tikhonov penalty that smooths the learned function [12]. Our sequence model is trained with a cross-entropy objective rather than squared error, so we read this as motivation rather than an exact equivalence. This discourages the model from staking predictions on fragile surface details. The effect is pronounced for text. Neural models are easily broken by small character-level corruptions such as typos and transpositions, and training on such noise is an effective and commonly used intervention for restoring robustness [13]. Simple edit-based augmentation, at the word or character level, is a lightweight and widely used instance of the idea [14]. The perturbations applied in this thesis (subsubsection 6.2.4) are one such scheme, shaped to the failure modes of hand-transcribed, multilingual historical text.

3.6 Evaluation metrics

Coding quality is measured with set-based multi-label metrics. For each record i , the decoder output and the true target are parsed into unordered sets of ICD10h codes, P_i and G_i , by splitting on the label separator and discarding whitespace and duplicates. Precision, recall, and their harmonic mean F1 are computed over these code sets in the standard way, with full definitions collected in Appendix F. We use three headline metrics, exact-match accuracy, micro-F1, and macro-F1, alongside the Jaccard similarity as a complementary set-overlap measure.

Exact-match accuracy. The fraction of records whose predicted set exactly equals the true set, $P_i = G_i$. It is the strictest metric, and in this multi-label setting it is identical to record-level accuracy.

Micro-F1. The F1 obtained by pooling the code-level true and false positives and negatives across all records before averaging. It is frequency-weighted, so it reflects overall operational performance but can mask poor results on rare codes.

Macro-F1. The unweighted mean of the per-code F1 scores, so every ICD10h code counts equally regardless of frequency. It is the primary metric of this thesis, because the ICD10h label distribution is long-tailed, so a model that learns only the common causes scores well on accuracy and micro-F1 yet poorly on macro-F1 [15].

Jaccard similarity. The mean over records of $|P_i \cap G_i|/|P_i \cup G_i|$, which rewards partial matches and sits between the all-or-nothing exact match and the code-pooling micro-F1.

Hierarchy-level scoring. ICD10h codes are hierarchical in the sense that the chapter letter gives the broad disease family, the following characters identify increasingly specific categories, and the final digits distinguish the full historical code. To distinguish near misses from entirely wrong predictions, we also truncate predicted and true codes to coarser levels and rescore them. A prediction with the correct chapter but wrong trailing digits is therefore counted as correct at the chapter level but wrong at the full-code level. subsection 7.1.1 reports this hierarchy-level evaluation from chapter-level scoring down to exact ICD10h code matching.

4 Related Work

Automated coding of historical, multilingual causes of death into the ICD10h scheme has, to our knowledge, not been the subject of a dedicated prior study. The most closely related work either applies a simpler method to the same kind of data, applies a comparable fine-tuned transformer to a different historical variable, or performs the analogous coding task on modern clinical text. A separate literature on selective prediction and uncertainty estimation underpins the deployment workflow. This chapter positions the thesis against each of these adjacent literatures in turn, rather than against a single directly comparable system.

4.1 Automated coding of historical causes of death

The work closest to ours in both task and data codes free-text causes of death from the Copenhagen Burial Register (1861–1911) using Jaro-Winkler string similarity, matching each historical string to the closest entry in a reference vocabulary and reaching roughly 50% accuracy [16]. It shares our problem shape and data lineage but differs in two important respects: it is a purely lexical method with no learned multilingual representation, and it targets the older DK1875 scheme rather than ICD10h. It is the simple prior alternative that a learned model should improve on.

The ICD-style coding of the same Copenhagen material has otherwise been a manual undertaking, documented for the 1880–1881 register as expert hand-coding that must reconcile historical diagnostic rationales with a modern classification [17]. This establishes that coding the dataset to an ICD-style scheme is, in current practice, slow expert labor rather than an automated step, which is the bottleneck this thesis targets. The ICD10h scheme itself [3, 4] provides the target vocabulary the coded data is expressed in.

The most direct precedent applies large language models to historical cause-of-death coding [18]. On a sample of nineteenth-century English-language UK death registers, it evaluates zero-shot GPT-3.5, GPT-4, and Llama 2 against classical supervised classifiers. Its strongest figures, GPT-4 at about 83% and a support-vector machine at about 89%, are not directly comparable to ours. The language-model figure mixes exact matches with three-character partial credit, so its true exact-match rate is 57.9%. The supervised figure is an in-distribution accuracy, and neither reports macro- or micro-F1. The study also codes to modern ICD10 rather than ICD10h, which was unavailable to its models, on English-only, single-cause records. We therefore read it as a motivating precedent rather than a benchmark, since it finds off-the-shelf zero-shot models inadequate for the

task and names fine-tuning and retrieval over an ICD10h reference list as the next steps, which is the direction this thesis takes, while additionally addressing the multilingual and multi-cause coding it set aside.

4.2 Automated standardization of historical records

The strongest methodological parallel is OccCANINE, which fine-tunes the tokenization-free CANINE encoder to map free-text historical occupational descriptions onto HISCO codes across thirteen languages, reaching roughly 96% accuracy [19]. It establishes that a fine-tuned multilingual transformer can reliably standardize noisy historical records into a fixed historical classification, and its character-level design is explicitly motivated by robustness to archival spelling and transcription variation. Our task shares this structure but targets a different variable, mapping historical causes of death onto ICD10h rather than occupations onto HISCO. The architectures also differ, in that OccCANINE classifies with a tokenization-free, character-level encoder (CANINE), whereas codLLM generates the code as text with a subword sequence-to-sequence model (FLAN-T5). The parallel is therefore at the level of the approach, the fine-tuned multilingual standardization of historical text, rather than identical architecture.

4.3 Automated clinical ICD coding

Automated ICD coding is well studied for modern clinical text. Attention-based neural models such as CAML [20] and label-attention architectures [21], and more recently pretrained-language-model approaches such as PLM-ICD [22], predict ICD codes from discharge summaries, typically benchmarked on the MIMIC-III critical-care dataset [23]. This line of work has itself been critically reviewed and reproduced [24]. These systems operate under assumptions that do not hold in the historical setting: contemporary clinical English rather than archaic multilingual text, abundant labeled training data rather than a scarce hand-coded resource, and a standardized terminology rather than scribe-specific vernacular and OCR noise. We therefore treat modern clinical ICD coding as an adjacent but substantially easier problem, useful for context but not directly transferable.

Generative formulations of ICD coding extend this line beyond the encoder models above. Generative models have been used for open-vocabulary and extreme multi-label classification, emitting the label set as text [25], and for ICD coding framed as autoregressive generation, where the generative view helps the rare and few-shot tail [26]. A recent system codes death certificates to ICD with a generative model and constrained decoding to guarantee valid outputs [27]. This thesis takes the same generative view, and the long-tailed label distribution that shapes several of its training choices is itself a well-studied problem in natural-language processing [28].

4.4 Selective prediction and uncertainty estimation

The deployment workflow this thesis proposes (RQ2) rests on selective prediction, in which a model may abstain on the inputs it is least sure of, trading coverage for higher accuracy on the records it does answer. The framework is established for deep networks, where deferring the least-confident predictions sharply raises accuracy on the rest [29]. The signal that drives the abstention is usually the model’s own output. The probability it assigns to its prediction is a strong baseline for separating correct from incorrect predictions [30]. Modern networks are, however, poorly calibrated, so absolute probabilities overstate confidence and need post-hoc correction such as temperature scaling [31]. Because our deferral ranks predictions by confidence rather than thresholding an absolute probability, it does not require the scores to be calibrated.

Because our model generates the code token by token, its uncertainty can be read straight from the decoding distribution, using the token log-probability and entropy signals defined in section 3.3, which are established for detecting errors in sequence generation [32]. We adopt them, calibration-free, to rank and defer predictions for expert review, applying selective prediction to historical, multilingual cause-of-death coding, where to our knowledge it has not been studied.

5 Data

This chapter describes the dataset used throughout the thesis, the preprocessing pipeline that produces train/val/test splits, and the most important properties of the data that later chapters will refer back to.

5.1 Data contract and ethical framing

The dataset is shared with the project under a signed data-use agreement between the project supervisors and the institutions that hold the original archival records. The agreement permits non-commercial academic research, including automated classification experiments, and forbids redistribution of the raw text outside the project. A summary of its terms is given in the appendix (Section B.1). Aggregated statistics may be published in this thesis, but the underlying text remains under access control. None of the datasets fall under the European Union’s General Data Protection Regulation for living individuals. The most recent year of death is 1939.

5.2 Sources

The dataset is a union of five hand-coded European historical mortality registers, totaling 1,525,048 records excluding the ICD10h masterlist (Section 5.5). The source-level record counts, languages, and year ranges are summarized in Table 5.1, and the records-per-year distribution is shown in Figure 5.1. Its pooled panel spikes at the 1918 influenza pandemic (39,339 records). The panels are drawn from the raw year columns and total 1,577,048. Preprocessing drops 52,000 of these rows, chiefly Copenhagen records that were never assigned a cause-of-death code and so carry no usable label, to reach the dataset total. Training also draws on the masterlist’s English reference strings as an auxiliary source (Section 5.5). The counts, table, and figures in this section describe the five registers evaluated in the thesis.

Throughout this thesis, the human-assigned ICD10h codes in these registers are treated as ground truth. This is a working assumption rather than a guarantee. The registers were coded by hand, so a small fraction of records are likely mislabeled through ordinary human error, and we do not have an independently re-coded subset with which to measure that rate. Any such label noise sets a practical ceiling on measured accuracy.

Each register is an independently assembled research dataset, and we credit its creators and providers here: Amsterdam [33], Copenhagen [5], Madrid [34], Belgium [35],

Source	Language	Records	Span (years)	Median year
Amsterdam (1853–1926)	Dutch	575,973	73	1890
Copenhagen (1854–1939)	Danish	501,855	85	1907
Madrid (1905–1927)	Spanish	366,282	22	1916
Belgium (1920–1930)	French / Dutch	42,011	10	1925
Ipswich (1871–1910)	English	38,927	39	1892
Total		1,525,048	86	

Table 5.1: Per-source record counts, languages, and year coverage and span.

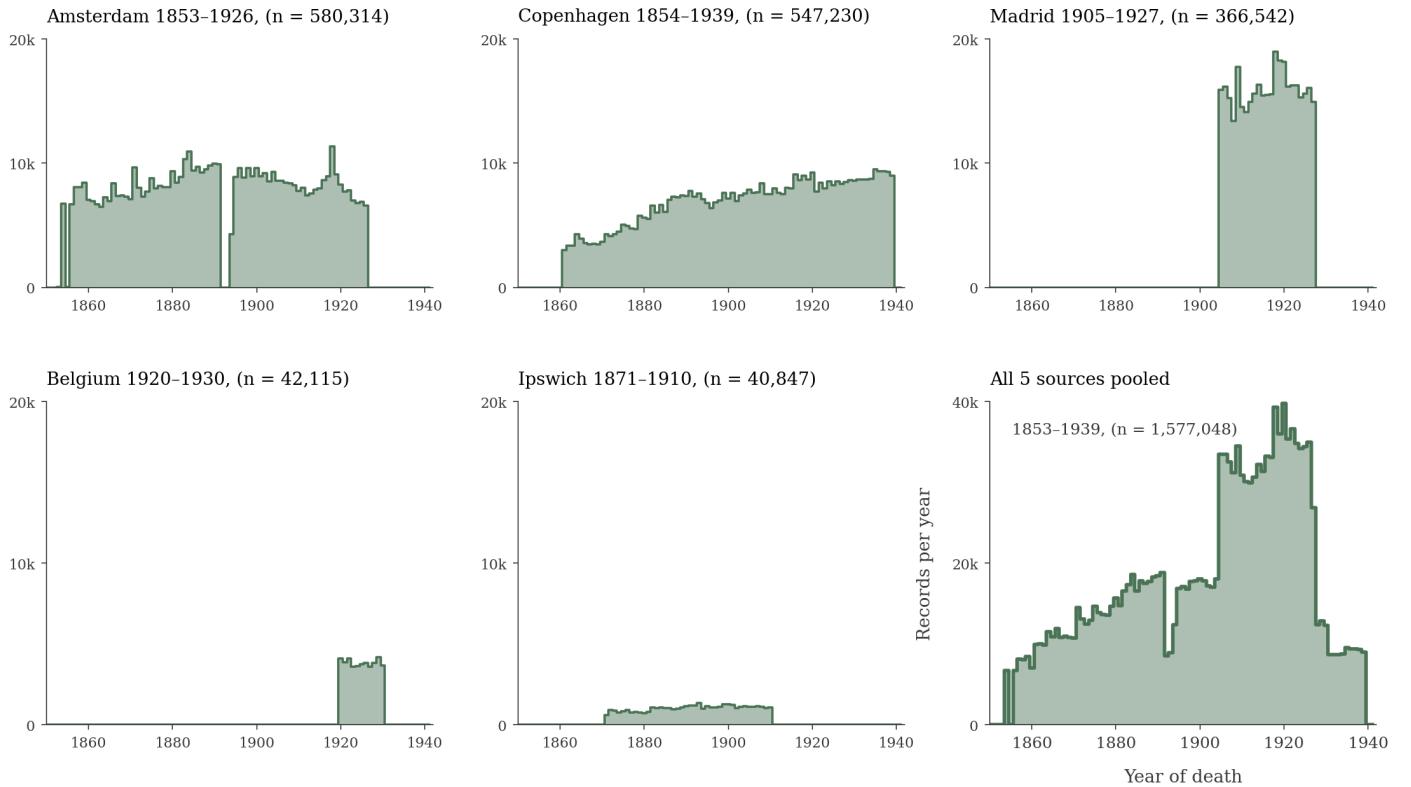


Figure 5.1: Records per year of death, per source (five panels) and pooled (bottom right).

and Ipswich [36]. We thank the individuals and institutions who assembled and shared these registers: Louise Villefrance Perner and the Copenhagen City Archives, with Mads Perner for providing access (Copenhagen); Angélique Janssens and the Radboud Group for Historical Demography and Family History at Radboud University, with Tim Riswick for providing access (Amsterdam); Eilidh Garrett and Alice Reid (Ipswich); Sven Vrielinck and Isabelle Devos of the Quetelet Center at Ghent University (Belgium, project S.O.S. Antwerp); and Diego Ramiro Fariñas, Stanislao Mazzoni, Mélanie Bourguignon, Begoña Villuendas Hijosa, and Michel Oris (Madrid). Each source was provided as a CSV, XLSX, or pipe-delimited text file, with a column schema agreed in the data-use agreement.

5.3 Descriptive statistics

The label space consists of all 14,088 possible ICD10h codes. However, of those, only 3,411 distinct codes appear in the dataset (1,465 of them in the held-out test split), with record frequencies ranging from 72,468 for the single most common code, J18.900 (unspecified pneumonia), down to single-instance codes (Figure 5.2). The distribution is long-tailed: 2,608 codes (76.4% of those observed) appear fewer than 50 times in the training data, and 825 of them appear exactly once. Only 206 codes reach 1,000 or more training records. Though just 6% of the distinct codes, they carry roughly 90% of all label occurrences, while the 2,608 rarest codes together account for only about 1%. Most of this long tail is the historical-extension surface of ICD10h: archaic disease names, region-specific conditions, and rare complications that did not survive into the modern ICD10 core vocabulary. This imbalance is what motivates macro-F1, which weights every code equally, as the primary metric.

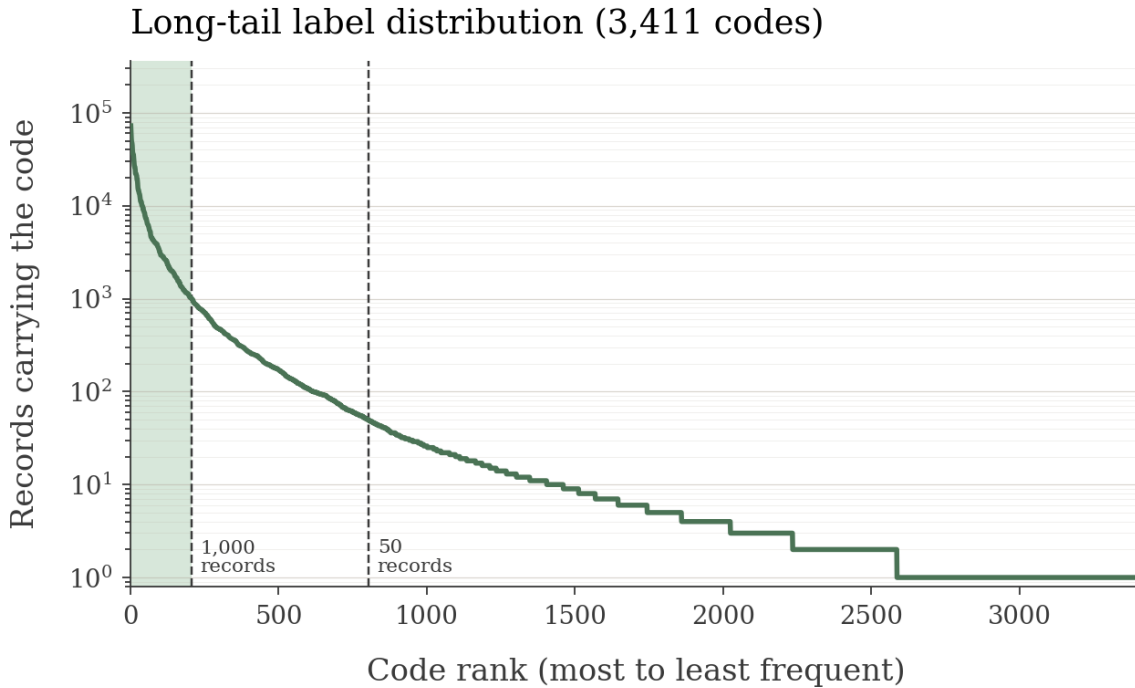


Figure 5.2: Rank-frequency distribution of the 3,411 distinct ICD10h codes (linear code rank, log record count). The two dashed lines mark the 1,000-record and 50-record thresholds. The shaded region is the high-frequency head above the first.

Multi-label distribution varies substantially across sources (Figure 5.3). Copenhagen records are entirely single-label because the Danish municipal coding practice attaches exactly one cause to each death. Madrid is predominantly single-label, Belgium and Amsterdam have a higher share of two-code records, and Ipswich is the strongest outlier, with 41.8% of records carrying two or more codes.

Aggregated to the 22 ICD10 chapters, the label distribution concentrates in a handful

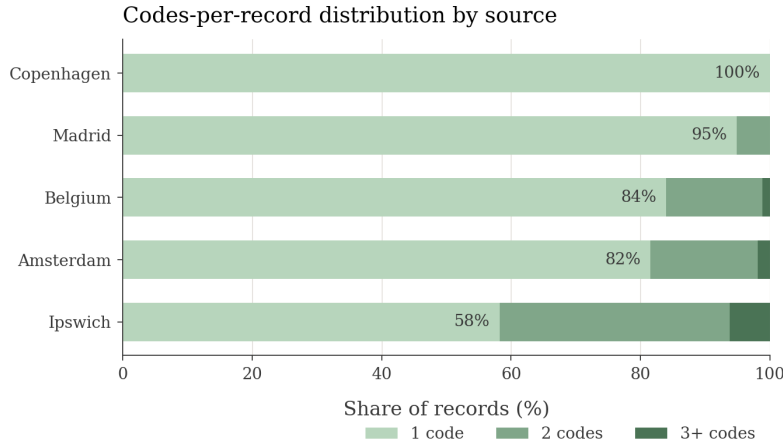


Figure 5.3: Codes-per-record distribution by source, as stacked horizontal bars shaded from lightest (one code) to darkest (three or more codes). The percentage in each bar is its single-code share, by which the sources are sorted.

of them (Figure 5.4). The five most prevalent are infectious and parasitic disease (**I**), respiratory (**X**), symptoms and signs (**XVIII**), circulatory (**IX**), and perinatal conditions (**XVI**). The prominence of the infectious, respiratory, and perinatal chapters is consistent with mortality in a period before antibiotics and with high infant mortality.

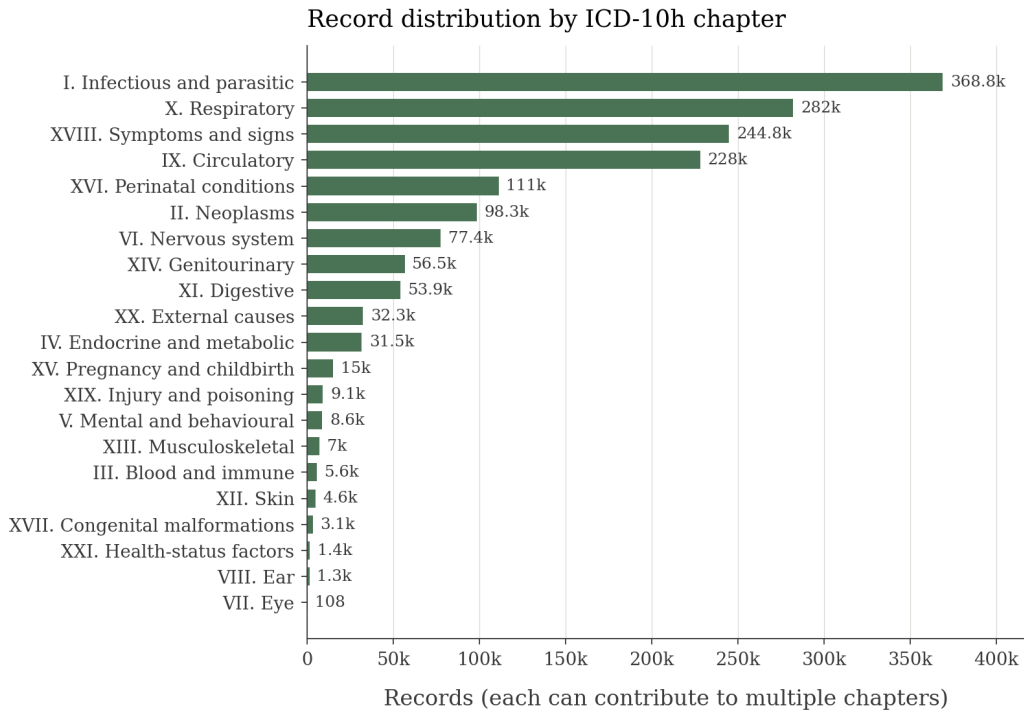


Figure 5.4: Record distribution across the 22 ICD10 chapters, sorted by descending record count. A record can count towards more than one chapter if it is multi-CoD.

Finally, a non-trivial fraction of records carries diagnostically uninformative text such as “unknown”, “natural causes”, “old age”, “senility”, and “found dead”. Counting these uninformative cause strings as a fraction of each source’s records, the share ranges from

6.5% in Belgium to 16.6% in Ipswich. These records remain codable because ICD10h provides ill-defined and administrative codes for them.

5.3.1 Source conventions and historical wording

The five registers are not interchangeable samples from one coding process. They also carry local language, transcription, and coding conventions. Table 5.2 gives selected examples that matter for interpreting later errors.

Pattern	Example wording	Dataset observation
Ill-defined causes	<i>unknown</i>	Madrid labels this wording as Z02.702, while Copenhagen and Ipswich label it as R99.001.
Local label convention	Amsterdam <i>diabetes</i>	Amsterdam uses E13.900, while the same plain wording is commonly mapped to E14.* in other sources.
Cardiovascular convention	Madrid <i>arteriosclerosis</i>	Madrid contains I38.001 examples, while similar atherosclerosis wording elsewhere is often under I70.*.
Local respiratory spelling	<i>bruncotaxia; bronquilectasia</i>	Rare source-specific spellings must still be mapped onto standardized ICD10h respiratory categories.
Mixed or damaged text	<i>Long tering, maladie contractée au front; Rup...? Exhaustion</i>	Some records combine languages, war context, or damaged transcription fragments in the same cause string.

Table 5.2: Examples of source conventions and historical wording in the historical registers. These examples describe properties of the data, not a taxonomy of model errors.

The point of these examples is that the model does not only learn medical terms, meaning it also learns how a source tends to attach local wording to ICD10h labels. These data nuances may impact the model’s understanding of said sources.

5.4 Splitting Strategy and Inherent Data Leakage

Splitting strategy We use a random 90/5/5 split with fixed data seed 333. Over the five historical sources this yields 1,372,533 training, 76,266 validation, and 76,249 test records (sum 1,525,048). The masterlist reference strings are split in alongside them as an auxiliary source (section 5.5), adding 2,975, 151, and 169 records respectively, so the full split including references is 1,375,508 / 76,417 / 76,418 (sum 1,528,343). The split

is computed once and persisted to a versioned cache. The per-source validation and test counts are within 4.88%–5.05% of the source totals.

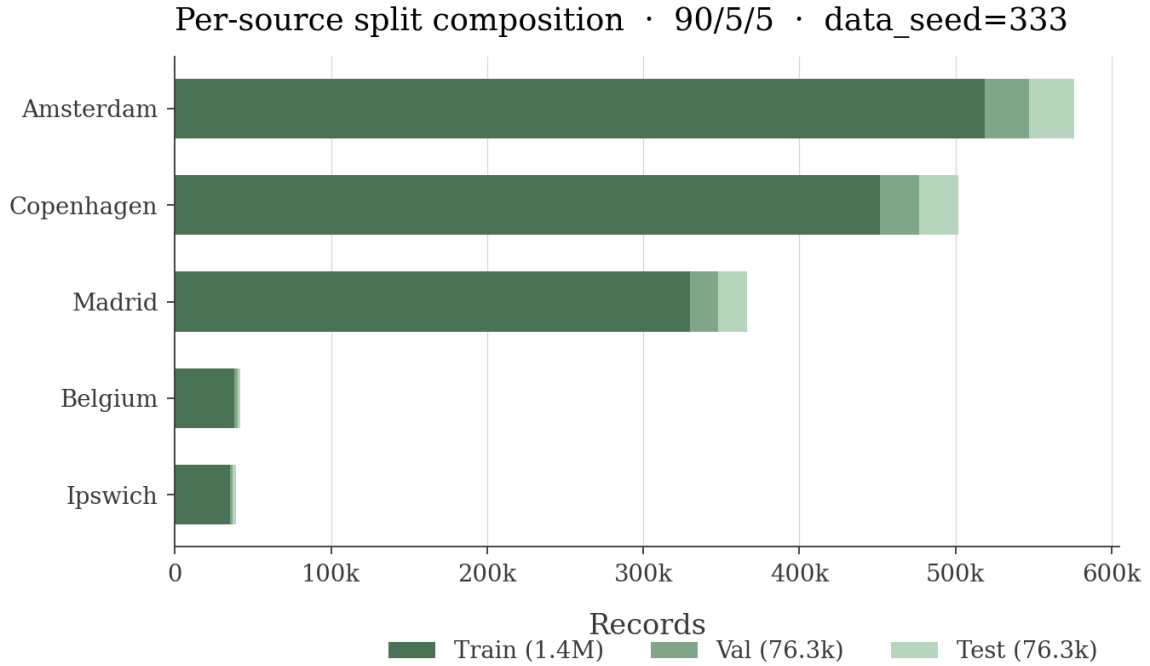


Figure 5.5: Per-source composition of the 90/5/5 train/val/test split. Each bar is a single source, segmented by the records that ended up in each split.

Data leakage arises because the dataset contains natural duplicates, meaning the same CoD string can appear hundreds or thousands of times within the same class. Therefore, when the dataset is split at the record level, identical strings inevitably end up in both train and test. Table 5.3 reports the cross-split overlap, where 92.19% of test records have text that appears verbatim in train and the normalized-match figure is 92.29%. We therefore report two test slices in Chapter 7:

- Full test (76,249 records): the record-level historical test, references excluded, including any duplicates that happen to fall on the test side. Chapter 7 reports headline metrics on this set.
- Unseen-string test (5,862 records, 7.69% of the pool): the subset of test records whose cause text never appears verbatim in training or validation. This is the slice that measures generalization rather than memorization.

The unseen-string slice is small and concentrated in the smaller sources: 36.4% of Ipswich test records, 16.0% of Belgium, 7.9% of Copenhagen, 6.4% of Amsterdam, and 5.1% of Madrid.

Pair	Records in B	Exact match (%)	Norm. match (%)	B unique strings
Train vs Val	76,266	91.97	92.09	15,540
Train vs Test	76,249	92.19	92.29	15,457
Val vs Test	76,249	84.73	85.01	15,457

Table 5.3: Proportion of text overlap between each pair of splits in the production 90/5/5 split. “Records in B” is the size of the second-named split. “Exact match” is the share of records in B whose text appears verbatim in the first-named split. “Normalized match” uses NFKD + diacritic strip + lowercase + whitespace collapse. “B unique strings” is the count of distinct text values in split B.

5.5 Masterlist and reference strings

The data contract delivers the ICD10h masterlist together with an English reference-string set, and the two play different roles. The *masterlist* is the classification reference, a table of 14,088 codes, each with its ICD10 mapping and an English description [37]. It defines the label space. The masterlist used here is the 2024 release, which revised 256 codes relative to the earlier October 2020 ICD10h version. It ships with a 2020-to-2024 transfer table mapping each revised code to its current form, which we apply to the register labels so that every code in the data resolves to a valid 2024 masterlist code (subsubsection 6.2.4). The *reference strings* delivered alongside the masterlist are 3,295 entries that map English cause-of-death phrases to 1,382 masterlist codes. They are loaded as an auxiliary source and split 90/5/5 alongside the five registers, so they contribute to training. Because they are clean dictionary mappings rather than archival text, the reported test metrics are computed on the historical records alone (76,249 of the 76,418 test records), so the references never inflate the headline numbers.

6 Methodology

6.1 Experimental Setup

6.1.1 Runtime, Versioning, and Compute

All experiments were implemented in Python 3.13 with Hugging Face Transformers [38]. Training and evaluation were tracked using Weights and Biases [39]. Dependency management used `uv` [40], with dependencies specified in `pyproject.toml` and resolved in `uv.lock`.

The project was version controlled using Git. Source code, experiment specifications, configuration files, tests, and documentation were tracked, while generated artifacts such as logs, processed datasets, checkpoints, local run folders, and cache directories were excluded. Experiment configuration was handled through checked-in TOML files under `runs/`. Before larger experiments, the expansion tool was used to inspect the concrete runs generated from a sweep specification. Correctness was checked through automated tests and linting.

All reported training runs were executed as LSF jobs on the DTU HPC cluster. Experiments were submitted using the repository’s `invoke` tasks. Generated job scripts loaded the run-specific environment, configured cache and output directories, synchronized the Python environment using `uv sync -frozen -no-dev`, and launched training with `uv run -no-dev python -m codllm.training`.

The main experiments used NVIDIA H100 GPUs through the DTU `gpuh100` queue, with one GPU per job in exclusive-process mode. The checked-in H100 profiles requested 17 CPU cores, allowing 16 PyTorch DataLoader workers plus the main training process. The cluster profile loaded CUDA 12.2. A representative H100 job environment is listed in section B.3. Separate H100 runtime profiles fixed sequence lengths at 256 input tokens and 32 output tokens, while the size-sweep specifications set per-device batch size and gradient accumulation so every size trained at an effective batch of 256 (Table 6.3). Generated data, outputs, tool caches, and the `uv` environment were placed under the configured run-storage directory.

6.1.2 Model and Fine-Tuning Contract

Every experiment used a FLAN-T5 checkpoint loaded from the Hugging Face Hub through `AutoModelForSeq2SeqLM`. The size sweep varies only the checkpoint: `google/flan-t5-small`, `-base`, `-large`, and `-xl` (Table 6.1). Models were full fine-tuned. All pretrained weights

were updated, with no adapters and no trainable-weight quantization.

Model	Hugging Face checkpoint	Parameters
codLLM-small	google/flan-t5-small	80M
codLLM-base	google/flan-t5-base	250M
codLLM-large	google/flan-t5-large	780M
codLLM-xl	google/flan-t5-xl	3000M

Table 6.1: The four FLAN-T5 checkpoints fine-tuned to produce `codLLM-small` through `codLLM-xl`.

Every size-sweep model was fine-tuned for up to 80 epochs at a learning rate of 2.5×10^{-5} with cosine decay, warmup ratio 0.1, no weight decay, and gradient-norm clipping at 0.5 (Table 6.2). Validation ran every epoch, the checkpoint with the highest validation macro-F1 was retained, and training stopped early once ten consecutive epochs failed to strictly improve it. Inputs and targets used 256 source tokens and 32 target tokens by default. The target budget is raised automatically if a multi-CoD target would otherwise not fit. `codLLM-base-deploy` shares the same 80-epoch, effective-batch, and checkpoint-selection contract, but differs from the size sweep on the three recipe factors listed in Table 6.4.

Hyperparameter	Value
Learning rate	2.5×10^{-5}
Max. epochs	80
LR scheduler	cosine
Warmup ratio	0.1
Weight decay	0
Gradient clipping (max. norm)	0.5
Effective batch size	256
Early-stopping patience	10 epochs
Early-stopping threshold	0 (any strict improvement)
Selection metric	validation macro-F1

Table 6.2: Fine-tuning hyperparameters shared by all four model sizes in the size sweep. Ablation experiments override individual rows as noted in chapter 7.

Model	Per-device batch	Grad. accumulation	Effective batch
codLLM-small	256	1	256
codLLM-base	128	2	256
codLLM-large	64	4	256
codLLM-xl	16	16	256

Table 6.3: Per-size batch composition for the size sweep. `codLLM-xl` was reduced to a per-device batch of 16 after an out-of-memory failure at 32.

Training used the Hugging Face `Seq2SeqTrainer` with its default AdamW optimizer and `lr_scheduler_type` schedule selection. The size sweep uses cosine decay, fixed before the scheduler ablation finished. That later ablation (section 7.3) found constant-with-warmup best at validation macro-F1 0.5595 against 0.4673 for cosine, so `codLLM-base-deploy` adopts constant-with-warmup, with the caveat that this eight-epoch result may not extrapolate identically to the 80-epoch run, since constant-with-warmup lacks the decay that cosine applies over long training. H100 runs used bfloat16 mixed precision, and evaluation used generation (`predict_with_generate`), so metrics are computed from decoded ICD10h strings rather than teacher-forced logits.

6.1.3 Data and Evaluation Protocol

Data split

The non-holdout comparisons use the 90/5/5 split described in chapter 5. Training uses the training set, validation drives checkpoint selection and early stopping, and the test set is reserved for final reported numbers. Selecting the deployment settings on the test split instead would be a form of data leakage, since it would tune the configuration against the held-out set and leave the final reported numbers no longer an unbiased estimate. Every ablation and model-selection decision therefore reports validation macro-F1, and the test split is touched only once, for the final model. Leave-one-source-out experiments rebuild this split after removing the held-out source.

Evaluation

Evaluation reports generated predictions on the five historical sources, excluding the masterlist reference strings (section 5.5). The primary metric is macro-F1. Micro-F1 is reported alongside it, together with exact-match accuracy. Because records may carry up to three codes, predictions and labels are compared as sets. We also report Jaccard similarity as a complementary set-overlap measure. Hierarchy-aware metrics additionally collapse codes to their three-character ICD10 category and chapter letter. As a secondary, descriptive check on whether accuracy tracks how often a chapter appears in the dataset, Appendix E breaks performance down by ICD10 chapter. Because the chapters are very unevenly populated (from roughly 18,000 records to a handful), that breakdown reports record-weighted exact-match accuracy and micro-F1, not macro-F1, which is unstable in the thinly populated chapters.

Metrics are reported on the full test set, the unseen-string slice (cause strings never in training or validation), per-source slices, and leave-one-source-out holdouts. Both the full-test and unseen-slice numbers are reported throughout. The unseen slice is the estimate of coding capability where memorization cannot inflate the result, while the full-test number

is the operational performance on this duplicated dataset, so neither alone is taken as the whole picture. The subgroup breakdown by sex, age band, and source is reported as exact-match accuracy rather than macro-F1, since per-subgroup label supports are too small for a stable macro average. Calendar year of death is not retained as a separate field in the processed dataset, so a finer year-by-year analysis is left to future work. RQ2 reports retained accuracy at a range of coverages after deferring the least-confident predictions to human review. The confidence signals are described in subsubsection 6.2.3.

Diagnostic error analysis

The qualitative results use the same generated predictions as the quantitative test evaluation. For compact inspection, every true and predicted ICD10h code is collapsed to its three-character category prefix, so errors are analyzed as broad category-level confusions rather than full leaf-code mistakes. This view is descriptive. It is used to understand failure modes after evaluation, not to select checkpoints or tune the model.

For each evaluation scope, three diagnostic summaries are computed. False negatives rank true category prefixes that the model misses, with the number of misses, the true support, the miss rate, and recall. False positives rank category prefixes that the model predicts incorrectly, with the number of false positives, the predicted support, the true support, the false-positive rate, and precision. Confusion pairs rank mismatched true prediction category sets, with their count, share of all mismatched rows, and one representative example showing the input, true code, and predicted code.

Confidence intervals

The historical test dataset is heavily duplicated. About 92% of test records share their exact cause-of-death string with another record, so the records are not statistically independent. Treating them as independent, as a normal-approximation or Bernoulli interval would, understates the sampling uncertainty. We therefore report confidence intervals from a cluster bootstrap whose resampling unit is the distinct input string rather than the individual record. We draw $B = 5000$ resamples of the distinct strings (seed 333), recompute each record-weighted metric on every resample, and report the result as a mean plus or minus half the width of the 2.5th-to-97.5th percentile interval. Exact-match accuracy, micro-F1, and Jaccard receive these intervals. Macro-F1 is reported as a point estimate with no interval, because it averages per-class F1 over a long tail of rare codes (501 of the codes observed in the test set appear exactly once), so a resampling interval for it is dominated by which singletons happen to be drawn rather than by genuine model uncertainty.

These intervals are deliberately narrow in scope. They capture the sampling and duplication uncertainty of one fixed model evaluated on one dataset under a single data

seed. They are a lower bound on the true uncertainty and do not license claims about variation across training seeds, data splits, or unseen sources. We report them because an honest interval on a duplicated dataset is more informative than a falsely precise independence-based interval or no interval at all. The agentic baselines are scored on smaller seed-333 subsamples that cannot populate the singleton-heavy rare-code tail, so the macro-F1 reported for them is restricted to that matched subset and is not comparable to a full-test macro-F1 (Table 7.5).

6.1.4 Reproducibility

Runs are reconstructed from their recorded configuration rather than ad-hoc command-line state. Runtime behavior is owned by a single configuration object set through checked-in TOML specs with environment-variable overrides. The Python version and dependencies are pinned through `pyproject.toml` and `uv.lock`. On the cluster, tool caches are redirected to a fixed storage location. Processed-data and prepared-split caches are keyed by metadata, so changes to raw-file signatures, source mappings, harmonization, split settings, balancing, or multi-CoD options force a rebuild. Randomness is governed by separate data and training seeds applied to Python, NumPy, and PyTorch, while low-level CUDA execution remains partly nondeterministic.

6.2 Methods

6.2.1 Pipeline and Output Contract

Output contract

The model is trained as a text-to-text ICD10h coder. The final runs use cause-of-death text plus age and sex, joined with field prefixes. Other metadata such as language is omitted. The target is one to N ICD10h codes, separated by the configured label separator. For evaluation, predictions and true targets are normalized, split into code sets, and compared as unordered labels, so missing, extra, duplicate, and malformed codes reduce the set-based metrics.

Pipeline architecture

Each run follows the same ordered pipeline. Configuration flags decide whether optional branches run:

1. For cluster runs, the generated LSF job exports the TOML-defined environment, prepares cache/output paths, synchronizes the locked `uv` environment, and starts `python -m codllm.training`. The entrypoint builds `Config` from environment

variables and CLI overrides, allocates or resumes a run-scoped output directory, and initializes the Weights & Biases run before data preparation begins.

2. The data handler builds metadata for the requested prepared splits and takes a file lock on the corresponding split-cache key. If a complete prepared-split cache exists, the cached train, validation, test, holdout, and holdout-evaluation dataframes are loaded directly. If not, the handler loads or rebuilds the processed-data cache under its own lock by reading enabled raw sources, applying source mappings and skip rows, building configured input fields, limiting rows by maximum label count, concatenating sources, applying label harmonization when enabled, and writing processed data plus metadata.
3. From the processed dataframe, the handler first removes any leave-one-source-out holdout source into a separate holdout dataframe. It then removes configured non-training sources, optionally subsamples the remaining pool by `dataset_size`, and splits that pool into train/validation/test with the configured data seed.
4. Prepared-split transformations are then applied. The training split can receive synthetic multi-CoD rows, while train, validation, test, and holdout labels can have multi-CoD order shuffled for order-invariant learning and evaluation. A sampled holdout-evaluation dataframe is built only when during-training holdout evaluation is configured. The training split then receives the balance policy: floor upsampling with perturbations on sampled upsample rows, followed by whole-training-set base perturbation when its rate is nonzero. If masterlist injection is enabled, upsampled and perturbed masterlist rows are appended after the balance policy. The resulting prepared splits and balance/injection diagnostics are cached and checked for source integrity.
5. If pretraining is enabled, the masterlist pretraining source is loaded separately from the historical prepared splits. It is upsampled per label, perturbed, and optionally augmented with synthetic multi-CoD rows before being used as the pretraining dataset.
6. Model setup happens after data preparation. For the thesis runs, the seq2seq model and tokenizer are loaded once; if pretraining is enabled, the same model is first trained on the masterlist stage and then continued on the historical fine-tuning stage. Each stage tokenizes its own dataframes, attaches the exact-match and multi-label metric functions, configures callbacks for optional sampled holdout evaluation, early stopping, time-budget saves, and SIGTERM saves, and trains with a stage-scoped `Seq2SeqTrainer`. The code also contains a sequence-classification branch, but the reported thesis models use the seq2seq branch.

7. After training, the final model is evaluated on the full test split. When a leave-one-source-out holdout exists, the full holdout source is evaluated separately. If end-of-training uncertainty evaluation is enabled for a seq2seq run, the model regenerates test predictions with token-level uncertainty signals and writes the selective-prediction artifacts.

6.2.2 Controlled Comparisons and Experiment Grid

Controlled comparisons

Most method choices are evaluated through controlled one-factor comparisons. In this thesis, an *ablation* means rerunning the same pipeline while changing one method component, such as pretraining, upsampling, perturbation, input fields, or the learning-rate schedule, and keeping the remaining configuration fixed. The size sweep is the exception. It varies the FLAN-T5 checkpoint while keeping the method contract fixed. The ablations form a sequence of single-factor comparisons, not a joint search over the full hyperparameter grid, and `codLLM-base-deploy` combines the per-factor best settings found rather than the result of a joint search (section 7.3).

The ablation runs are short because the compute budget did not allow more. Each one-factor sweep ran for eight fine-tuning epochs on a single seed (333), against the 80-epoch budget of the size sweep and deployment run. They are reported on validation macro-F1, the metric that also selects each checkpoint, so they are descriptive within-sweep evidence rather than headline test numbers. Their absolute scores (roughly 0.44 to 0.56) are not comparable to the 80-epoch size sweep, so the comparison that matters is between cells of the same sweep, which share the `base_thesis` reference cell at validation macro-F1 0.4673. The single-seed results are not subjected to significance testing. Seed variance is not the object of study here, so differences are reported and their direction read rather than treated as statistical claims. The size sweep itself was started with the best settings known at that time (upsampling floor 50, synthetic multi-CoD ratio 0, cosine schedule, base perturbation rate 0.5, cause text with age and sex, masterlist pretraining on), and `codLLM-base-deploy` adopts the per-factor best values found by the ablations rather than these initial defaults, except the perturbation rate, which is held at 0.5 as an unseen-string robustness hedge rather than the validation-best 0.0 (Table 7.7). The size sweep could not wait for the ablations to finish. Training all four sizes, `codLLM-x1` alone taking on the order of weeks, had to start before the single-factor sweeps completed, so it was frozen at the settings best known when it started. Re-running the full four-size sweep with the later ablation winners was not affordable under the compute budget, so only `codLLM-base-deploy`, and the leave-one-source-out transfer runs built from it, adopt them (Table 6.4).

Setting	codLLM- <code>{size}</code>	codLLM-base-deploy
Upsampling floor	50	200
Synthetic multi-CoD ratio	0	0.30
LR scheduler	cosine	constant-with-warmup
Base perturbation rate	0.50	0.50
Input fields	CoD + age + sex	CoD + age + sex
Masterlist pretraining	on	on
Max. epochs	80	80

Table 6.4: Size-sweep recipe versus deployment recipe. The columns document two experimental roles, not a controlled head-to-head: `codLLM-{size}` isolates model capacity under the recipe available when the sweep started, while `codLLM-base-deploy` adopts the later per-factor ablation winners. Three settings differ, shown in bold. The rest of the contract is shared. The leave-one-source-out deployment transfer runs use the deployment recipe.

Experiment grid

The method choices are backed by checked-in TOML specifications under `runs/`. Unless noted otherwise, they inherit the same reference contract from the `base_thesis` specification: the full dataset, a 90/5/5 split, eight fine-tuning epochs, validation macro-F1 checkpoint selection, max three labels, floor upsampling to 50, base perturbation rate 0.5, no masterlist pretraining, and no final uncertainty pass. The ablations are drawn on the base architecture because it is a faithful stand-in for the hardware a historical demographer would realistically run, a mid-sized model (≈ 250 M parameters) that fine-tunes and runs on a single accessible GPU, so the method is measured where it is most likely to be applied. Fixing the model for the ablations is also a matter of compute. A full ablation grid rerun at every model size would multiply an already large set of training runs by four, and the largest models are precisely the ones that cannot be run many times, so holding the model fixed spends the budget on covering more methodological factors instead of re-confirming each at four sizes. The obvious caveat is that a factor’s effect at base may not be identical at every size. The conclusions assume that the factors scale similarly across sizes, since the full cross-product of factor and size could not be run, and cross-term effects across all tests were not exhaustively tested.

Specification	Varied factor	Levels	Purpose
Reference and scaling			
base_thesis	none	fixed reference cell	reference contract
model_size	codLLM size	small, base, large, xl	model scaling
Training recipe			
scheduler	LR schedule	cosine, linear, constant-warmup	schedule choice
input_features	input fields	CoD, CoD+age+sex	metadata effect
Data augmentation			
upsampling_floor	rare-code floor	0, 25, 50, 100, 200	floor choice
perturbation_rate	base perturbation rate	0, 0.25, 0.50, 0.75, 1.00	noise strength
multicod_synthetic	synthetic multi-CoD ratio	0, 0.15, 0.30	multi-CoD synthesis
Masterlist and transfer			
masterlist_injection	fine-tuning masterlist rows	off, 10, 20 per label	direct injection
pretraining	masterlist pretraining	off, on	pretraining effect
holdout	held-out source	five historical sources	source transfer
holdout_pretraining	holdout with pretraining	five historical sources	transfer with pretraining

Table 6.5: Controlled experiment specifications used to justify method choices. Specification names refer to checked-in TOML files under `runs/`.

6.2.3 Baselines and Selective Prediction

Baselines

The baseline methods use the same output contract as the fine-tuned model. Each method returns one to three ICD10h codes per record, and predictions are scored with the metrics in subsection 6.1.3. Full-split baselines (the TF-IDF classifier, the multilingual-embedding retriever, and the majority floor) are evaluated on the entire locked historical test split. The agentic baseline, being far costlier per record, is run on a seed-333 sample and reported as a sampled result in subsection 7.2.3.

The agentic baseline classifies each record with a fresh, isolated large-language-model agent that has tool access to a shell, the open web, and the ICD10h masterlist, and must return its codes in a fixed JSON schema. We deliberately grant this full tool access, including the open web, rather than sandboxing the agent, because the goal is to test a strong contemporary agent at its most capable. Any deficit then reflects the task rather than a handicapped baseline. We run it with two models under the identical prompt and tool configuration, the cost-efficient Claude Sonnet 4.6 and the frontier Claude Opus 4.8, so the comparison isolates model capability from prompt design. The prompt opens by fixing the role and task:

*You are an expert ICD10h medical coder for historical cause-of-death records.
Your task is to classify one cause-of-death string, which may describe one or several causes, into between one and three ICD10h codes.*

and then supplies the code format, the masterlist schema and search guidance, the multi-cause output convention, and the JSON output contract. The full prompt is reproduced

in Appendix C. Each record is appended beneath the prompt with its cause text and, in the metadata setting, the recorded age and sex. Each agent is run on a seed-333 sample of the test split in both input settings. All reported agentic results (Table 7.5) use the 3,000-record subset common to the four runs, reference strings excluded, so no method is advantaged by an easier draw. Each baseline is evaluated under both input settings and reported at its stronger one. The classical and agentic baselines are stronger on cause text alone and reported there.

Compute budget for the agentic baseline

The agentic baseline is far more expensive per record than the other methods, which is why it is run on a sample rather than the full test split. Cost is measured directly from the run logs. Every agent call reports the API-equivalent price of the tokens it consumed through the command-line client’s `total_cost_usd` field, which we sum over the matched 3,000-record subset to give the per-configuration totals in Table 7.5. Each record takes 25 to 30 seconds. The resulting cost, its extrapolation to the full dataset, and what it implies for deployment are reported in subsection 7.2.3 and discussed in chapter 8.

Selective prediction

The selective-prediction results are reported in subsection 7.1.2. After training, the model regenerates each test prediction with greedy decoding and stores summed token log-probability, mean token log-probability, minimum token log-probability, mean token entropy, and first-token entropy. Predictions are ranked by each signal and the retained-set accuracy is evaluated at fixed coverages: 100%, 95%, 90%, 80%, 70%, 60%, and 50%. The curves are rank-based, so they require no probability calibration. An absolute calibration measure (expected calibration error via a validation temperature fit) is left to future work.

6.2.4 Training Data Interventions

Masterlist use

The ICD10h masterlist is used as the coding authority, not as another historical dataset. In training, it enters through supervised pretraining or through the separate masterlist-injection ablation. Reference strings are handled as data (section 5.5).

Label harmonization

The masterlist is the 2024 ICD10h release, and the data is harmonized to its label space before any other intervention. During loading, the pipeline reads the masterlist’s valid codes together with the 2020-to-2024 transfer table delivered in the same workbook, which

maps each of the 256 revised codes to its current form. Every record label is normalized to canonical ICD10h shape, remapped through the transfer table when it appears there, and then checked against the masterlist. A record whose labels do not all resolve to valid masterlist codes is dropped rather than kept with an invalid target. The same harmonization is applied to the masterlist pretraining data. This keeps the ground-truth labels consistent with the 14,088-code label space that defines them.

Multi-CoD handling

Multi-cause rows are kept as single examples whose target contains multiple ICD10h codes. Since evaluation treats these codes as an unordered set, the pipeline shuffles multi-CoD label order during split preparation. The pipeline can also synthesize multi-CoD rows by combining single-cause rows. It samples the number of codes from the observed multi-CoD cardinality distribution, draws distinct single-code examples, joins their text with a sampled separator, and uses the union of their labels as the target. This synthetic generation is disabled in the size sweep (ratio 0) and enabled for masterlist pretraining. `codLLM-base-deploy` enables it at ratio 0.3, the ablation best (section 7.3).

Upsampling

The training distribution is steeply long-tailed (chapter 5), so rare codes are upsampled during training. Every code with fewer than a target number of training records is lifted to that floor by resampling rows from the same code, while codes already above it are left untouched. The reference contract uses a floor of 50 with decay switched off, applied to single-label records only. Compound rows are left to the multi-CoD procedure, and resampled copies are perturbed in proportion to how hard their class was upsampled. The floor sweep (section 7.3) found a floor of 200 best at validation macro-F1 0.5202 against 0.4673 at floor 50, so `codLLM-base-deploy` adopts floor 200.

Perturbation

Character-level noise is injected only into training cause-of-death text. Four edit operations are used: swapping adjacent characters, deleting a character, substituting a QWERTY neighbor, and replacing a vowel with an accented variant. These operations are used as plausible character-level regularization rather than as an experimentally selected noise model. Training with input noise is a long-standing regularization technique [12, 13, 14]. The input carries no language tag in the final runs. Whether an explicit language tag would help the model handle multilingual spelling variation, especially on unseen sources, was not tested here and is deliberately left to future work.

Noise enters in two places: upsampled copies are perturbed in proportion to their duplication factor, and a base pass perturbs a fixed fraction of the whole training split.

For a cause string of L characters, the number of edits is drawn from a Gaussian with mean mL and variance vL , rounded, clamped to $[1, L]$, and capped at $mL + 3\sqrt{vL}$. The upsampled copies use $m = 0.05$, $v = 0$. The final method uses a base perturbation rate of 0.5. The rate sweep (section 7.3) found rate 0 best on validation macro-F1, but a moderate rate of 0.5 is kept as a robustness hedge for unseen strings rather than to maximize in-distribution macro-F1.

Pretraining

The pretraining stage is supervised intermediate training, not training from scratch. When enabled, the same FLAN-T5 checkpoint is first trained on the ICD10h masterlist before historical fine-tuning. Each pretraining example uses the configured masterlist description field as input and the corresponding ICD10h code as target. Supplementary training on a related task before fine-tuning is a standard transfer-learning technique that generally improves downstream and cross-domain performance [41, 42].

The pretraining data is loaded separately from the historical train/validation/test split. It is upsampled to at least eight examples per label. Added copies are perturbed with six character-level edits drawn from the same operation set. In the size sweep and `codLLM-base-deploy`, synthetic multi-CoD pretraining rows are generated with ratio 0.15. The stage runs for 16 epochs with learning rate 1×10^{-5} , a constant learning-rate schedule, warmup ratio 0.5, and validation every four epochs on the regular validation split. After this stage, the same model weights continue into the normal fine-tuning stage.

6.2.5 Discarded Alternatives

Several plausible alternatives were considered but not used in the final method. The most important architectural alternative was a flat classifier head over ICD10h codes. This would work naturally for a single-label task, but multi-CoD records would require a multi-label classifier with one output score for every ICD10h code and a threshold rule for accepting all codes above that threshold. With 14,088 possible codes and a long-tailed label distribution, that threshold would be hard to calibrate. Too high would miss secondary causes and rare labels, while too low would produce many spurious codes. The seq2seq formulation avoids this by generating one to several codes directly and by sharing output-token structure across related ICD10h codes.

We also explored prompting an LLM to traverse the ICD10h hierarchy and select a code through a sequence of tree decisions. This was dropped because each wrong branch decision removes the correct code from later consideration, and the method turns one prediction into many brittle intermediate decisions. An exploratory probe with a small local model (Qwen2.5-7B) made the failure concrete. At full depth the model would not follow the required structured output format on essentially any call, collapsing the search

to a near-random walk at roughly ten model calls and 37 seconds per record, so the run was abandoned after seven records. Even when the correct ICD10 chapter was supplied for free, accuracy then fell at every step down the tree, from 47.5% at the first level below the chapter to 22.5%, 15.0%, and 12.5% at the successively finer levels down to the full ICD10h code, with about 60% of all errors committed at the very first step below the chapter. An early wrong turn is unrecoverable, so the deep hierarchy multiplied the error rather than containing it.

A retrieval-augmented variant was also considered, retrieving or pruning candidate ICD10h entries from the masterlist and asking a model to choose among them. This was not adopted because the retrieval step can discard the correct rare or archaic code before the model sees it, especially under multilingual spelling variation, and because the extra retrieval machinery did not match the simpler end-to-end training objective. A direct measurement on a sample of the Belgian records, embedding both the cause strings and the masterlist entries with a multilingual sentence encoder, showed this retrieval recall ceiling to be the binding limit. The nearest 25 candidates held the full true-code set for only about two-thirds of records, and widening the pool to 200 candidates, out of roughly 14,000 codes, still left close to one in five records missing a correct code. Since a code the retriever drops cannot be recovered downstream, this ceiling caps the entire pipeline, and the misses concentrate on exactly the historical, multilingual strings the method most needs to handle. The Dutch tuberculosis term *long tering*, for example, retrieved obstetric codes for prolonged labor rather than the tuberculosis category, because the encoder read the Dutch *long* (lung) as the English word *long*.

7 Experiments and Results

Unless noted, every result is on the locked held-out test split (chapter 5, `data_seed=333`), with the English reference strings excluded, reporting exact-match accuracy, Jaccard, macro-F1, and micro-F1.

7.1 Model capacity and the deployment recipe

Table 7.1 reports held-out test performance for the four `codLLM` sizes under the controlled size-sweep recipe, and for `codLLM-base-deploy`, the base-size model trained with the final deployment recipe. `codLLM-base-deploy` is not a fifth capacity point: it shares the base architecture but raises the rare-code upsampling floor to 200, adds synthetic multi-CoD rows at ratio 0.3, and switches to a constant-with-warmup schedule (Table 6.4). The column is included to compare the final recipe with the controlled base-size size-sweep recipe.

Metric	codLLM (size-sweep recipe)				Deployment
	small	base	large	xl	base-deploy
<i>Full test</i> (76,249 records)					
Exact-match accuracy	0.977 ± 0.001	0.985 ± 0.001	0.986 ± 0.001	0.987 ± 0.001	0.985 ± 0.001
Jaccard	0.983 ± 0.001	0.988 ± 0.001	0.990 ± 0.001	0.990 ± 0.001	0.989 ± 0.001
Macro-F1	0.657	0.723	0.759	0.765	0.749
Micro-F1	0.979 ± 0.001	0.986 ± 0.001	0.988 ± 0.001	0.988 ± 0.001	0.987 ± 0.001
<i>Unseen-string slice</i> (5,862 records)					
Exact-match accuracy	0.773 ± 0.011	0.833 ± 0.010	0.851 ± 0.009	0.855 ± 0.009	0.843 ± 0.009
Jaccard	0.834 ± 0.008	0.876 ± 0.007	0.888 ± 0.007	0.891 ± 0.007	0.884 ± 0.007
Macro-F1	0.510	0.589	0.637	0.654	0.625
Micro-F1	0.855 ± 0.007	0.893 ± 0.006	0.906 ± 0.006	0.909 ± 0.006	0.902 ± 0.006

Table 7.1: Held-out test performance of the four `codLLM` sizes (size-sweep recipe) and `codLLM-base-deploy` (deployment recipe) on the five historical sources, reference strings excluded, on the genuine held-out test split. The $\pm$ values are 95% cluster-bootstrap half-widths over distinct input strings (5,000 resamples). Macro-F1 is a point estimate: 501 of the 1,465 codes observed in the test set have support one, so no confidence interval is reported. `codLLM-base-deploy` shares the base architecture; its column reports the deployment-recipe gain rather than a further capacity step.

From small to xl, full-test macro-F1 is 0.657, 0.723, 0.759, and 0.765. At the same base architecture, the deployment recipe scores 0.749 macro-F1, 0.985 accuracy, and 0.987 micro-F1. On the unseen-string slice, macro-F1 ranges from 0.510 to 0.654 across the size sweep and is 0.625 for `codLLM-base-deploy`.

7.1.1 Coding specificity

ICD10h codes are hierarchical, where an ICD10 chapter contains three-character categories, each category contains four-character ICD10 codes, and a final historical part names the specific cause [4]. Figure 7.1 reports macro-F1 with predicted and true codes truncated to three representative levels of the hierarchy, for the four sizes and `codLLM-base-deploy`. The full six-level breakdown is in Table G.1.

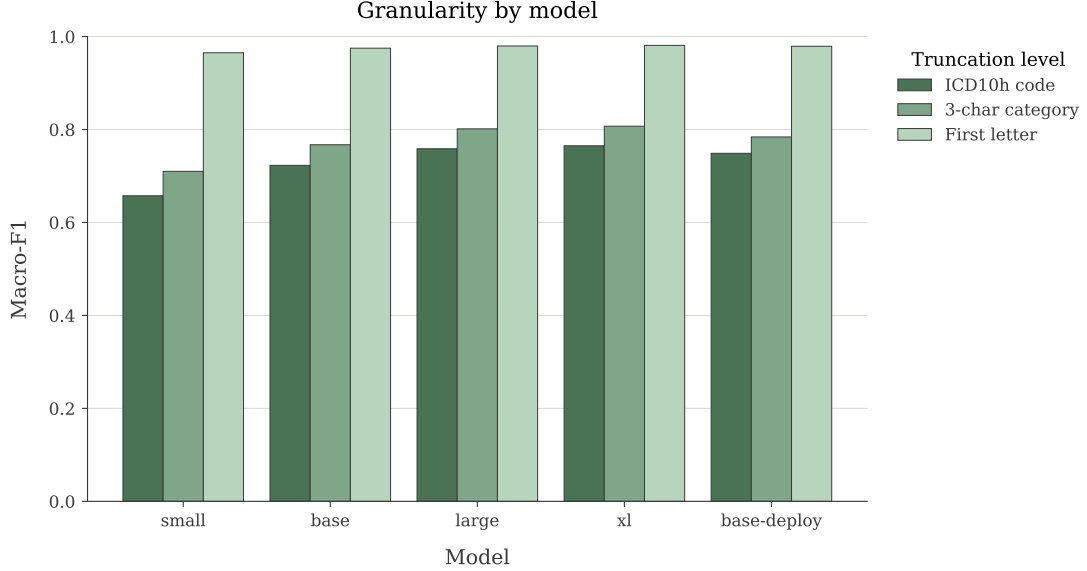


Figure 7.1: Macro-F1 on the full test set at three truncation levels of the ICD10h hierarchy (full ICD10h code, three-character category, and first letter) for each `codLLM` size and for `codLLM-base-deploy`. The full six-level breakdown, with exact values, the example code, and the class counts n that macro-F1 averages over, is given in Table G.1.

Across truncation levels, `codLLM-base-deploy` macro-F1 is 0.979 at the first letter, 0.784 at the three-character category, and 0.749 at full ICD10h specificity. The corresponding size-sweep base values are 0.975, 0.767, and 0.723.

7.1.2 Selective prediction

Figure 7.2 plots retained exact-match accuracy by coverage after ranking full-test predictions by mean token entropy (subsubsection 6.2.3). Every model exceeds 0.996 retained accuracy by 90% coverage and 0.999 by 80% coverage, with area under the risk-coverage curve below 0.0012 for all five.

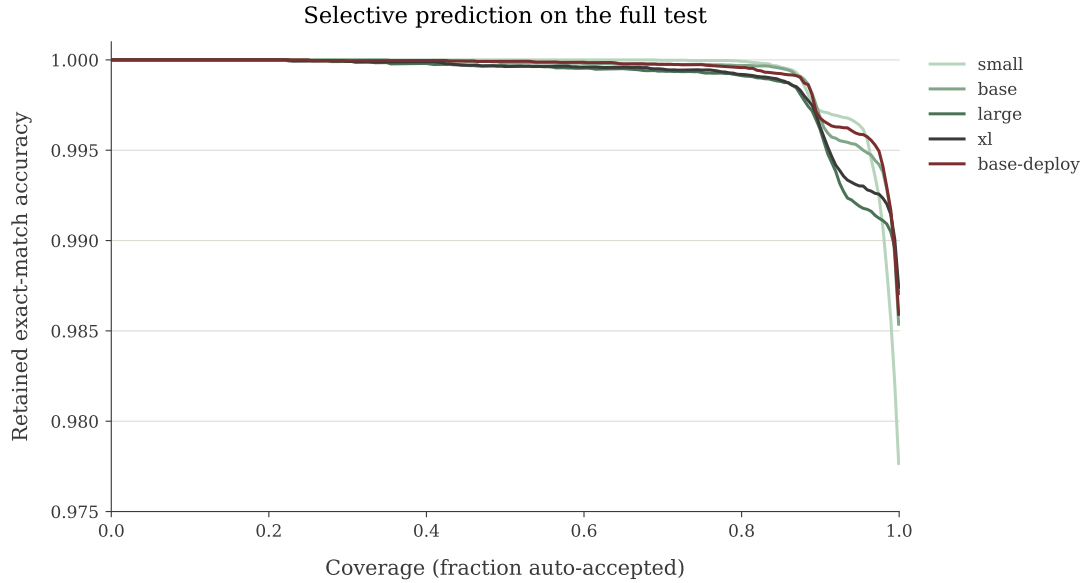


Figure 7.2: Retained exact-match accuracy against coverage on the full test for the four `codLLM` sizes and `codLLM-base-deploy`, ranking predictions by mean token entropy and retaining the most confident first. Coverage is the fraction of predictions auto-accepted. The rest are deferred. All curves remain above 0.985 over the plotted coverage range.

On the 5,862 unseen-string records, `codLLM-base-deploy` retained accuracy rises from 0.843 at full coverage to 0.925 at 80% coverage and about 0.94 at lower coverage (Figure 7.3).

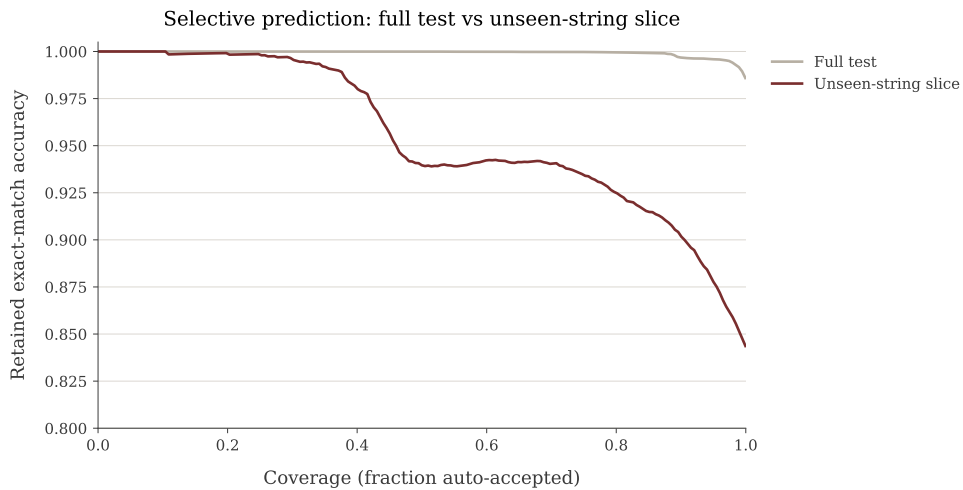


Figure 7.3: Selective prediction for `codLLM-base-deploy` on the full test ($n = 76,249$) and the unseen-string slice ($n = 5,862$, cause strings never in training or validation), ranking predictions by mean token entropy and retaining the most confident first. Full-test retained accuracy stays above 0.985; unseen-slice retained accuracy rises from 0.843 at full coverage to 0.925 at 80% coverage and about 0.94 with further deferral.

7.2 The deployment model

The remaining in-distribution analyses all use `codLLM-base-deploy`, the deployed model. On the historical test set ($n = 76,249$) it reaches 0.985 exact-match accuracy, 0.749 macro-F1, 0.987 micro-F1, and 0.989 Jaccard (Table 7.1).

7.2.1 Qualitative error patterns

Table 7.2 reports a compact set of representative category-level error families from the `codLLM-base-deploy` test predictions (subsubsection 6.1.3).

View	Category or pair	Count	Denominator or example
Missed category	R99	40	2,334 true (1.7% miss rate)
Missed category	J18	18	7,014 true (0.3% miss rate)
Missed category	C80	15	324 true (4.6% miss rate)
Over-predicted category	J18	17	7,013 predicted (0.2% FP rate)
Over-predicted category	I51	17	4,342 predicted (0.4% FP rate)
Confusion pair	E13 $\rightarrow$ E14	8	diabetes
Confusion pair	Z02 $\rightarrow$ R99	5	unknown
Confusion pair	I38 $\rightarrow$ I70	4	arteriosclerosis

Table 7.2: Representative category-level error diagnostics from the `codLLM-base-deploy` test output. Category rows use the same chapter-block false-negative and false-positive definitions as the W&B error tables. Pair rows group mismatched true and predicted chapter-block sets. Counts are small relative to the full test set.

In this diagnostic view, the largest missed category is R99 (40 misses), followed by J18 (18) and C80 (15). The most frequent confusion pair is E13 $\rightarrow$ E14 (8 instances).

7.2.2 Subgroup biases

Table 7.3 breaks `codLLM-base-deploy` exact-match accuracy down by recorded sex, age band, and source, on the full test and the unseen slice (cause strings never in training).

Subgroup	n	Full-test acc	Unseen-slice acc
<i>Sex</i>			
Male	38,145	0.986	0.847
Female	37,746	0.985	0.840
Missing	358	0.997	n/a
<i>Age band</i>			
<1	11,846	0.994	0.855
1–4	6,638	0.988	0.826
5–14	2,804	0.978	0.798
15–44	12,881	0.975	0.802
45–64	13,860	0.979	0.832
65–84	15,816	0.987	0.886
85+	1,819	0.992	0.882
Unknown	10,585	0.994	0.888
<i>Source</i>			
Copenhagen	25,128	0.999	0.996
Amsterdam	28,841	0.987	0.834
Madrid	18,100	0.974	0.601
Belgium	2,142	0.938	0.666
Ipswich	2,038	0.936	0.843

Table 7.3: Exact-match accuracy of `codLLM-base-deploy` by subgroup, on the full test and the unseen-string slice (cause strings never in training). n is the full-test subgroup size, and each block sums to the 76,249 reference-excluded test records. The missing-sex group keeps only one record on the unseen slice, so its unseen-slice accuracy is not reported.

7.2.3 Comparison with baselines

Table 7.4 compares the deployment model against the TF-IDF classifier, the MiniLM retriever, and the majority-class floor. The two agentic models are compared separately, on a matched subset, in subsection 7.2.3.

Method	Macro-F1	Micro-F1	Accuracy
<code>codLLM-base-deploy</code>	0.749	0.987	0.985
TF-IDF + linear SVM	0.314	0.532	0.007
Multilingual MiniLM cosine	0.041	0.179	0.000
Majority class	0.0001	0.044	0.039

Table 7.4: Macro-F1, micro-F1, and exact-match accuracy on the held-out test split, every method scored on the full split ($n = 76,249$, reference strings excluded). `codLLM-base-deploy` is shown with cause-of-death text plus age and sex; training-free baselines are shown with cause text alone. The two agentic baselines are compared against `codLLM-base-deploy` separately, on a matched 3,000-record subsample, in Table 7.5. The best value in each column is in bold.

The three metrics diverge sharply for the TF-IDF baseline, which reaches a moderate macro-F1 (0.314) and micro-F1 (0.532) but near-zero exact-match accuracy (0.007). The one-vs-rest classifier over-predicts, recovering many of the correct codes, which the F1 metrics credit, but rarely producing exactly the code set a record requires, which exact-match demands.

Agentic models on a matched subset

Table 7.5 scores both agents, in both input settings, against `codLLM-base-deploy` on one common 3,000-record subset.

Method	Input	Accuracy	Micro-F1	Macro-F1	Cost (\$ USD)
<code>codLLM-base-deploy</code>	CoD + age + sex	0.983	0.984	0.897*	≈ 0
Agentic Opus 4.8	CoD only	0.588	0.621	0.336	430
Agentic Opus 4.8	CoD + age + sex	0.560	0.596	0.323	492
Agentic Sonnet 4.6	CoD only	0.601	0.633	0.332	202
Agentic Sonnet 4.6	CoD + age + sex	0.560	0.596	0.298	252

Table 7.5: All methods scored on the same 3,000 test records, the largest subset common to both agentic models across both input settings, reference strings excluded, drawn at random and source-proportional (Amsterdam 38.7%, Copenhagen 31.4%, Madrid 23.5%, Ipswich 3.4%, Belgium 3.0%). The supervised row is `codLLM-base-deploy`, the deployment model. *The macro-F1 here (0.897) runs higher than its full-test value because this small subset holds far fewer of the singleton rare codes. It is kept on the matched set only for a fair head-to-head and is not its realistic macro-F1, which is 0.749 on the full 76,249-record test (Table 7.1). The cost column is the API-equivalent total of scoring all 3,000 records with that agent in that input setting (subsubsection 6.2.3). The best value in each column is in bold.

7.3 Configuration ablations

The one-factor ablations below each vary a single factor from the reference contract (subsubsection 6.2.2). Its cause-text-only cell scores validation macro-F1 0.4673, the reference value recurring in every table. Each sweep runs for eight fine-tuning epochs on a single seed at the base size, so the absolute scores (roughly 0.44 to 0.56) are not comparable to the 80-epoch headline numbers. The comparison that matters is between cells of the same sweep.

The upsampling-floor sweep (Table 7.6) is best at floor 200, with validation macro-F1 rising monotonically above floor 25.

Upsampling floor	Val. macro-F1
0	0.4411
25	0.4352
50	0.4673
100	0.4872
200	0.5202

Table 7.6: Effect of the rare-code upsampling floor on validation macro-F1 (base size, eight epochs, single seed). The best cell is in bold.

The perturbation-rate sweep (Table 7.7) is best at rate 0.0 on clean validation macro-F1, with the score falling as the rate rises. Character-level perturbation is intended as a robustness hedge against the spelling and transcription noise of unseen strings rather than a clean-text gain, so the deployment recipe retains a moderate rate of 0.5 rather than the validation-best 0.0, accepting a small validation cost for expected robustness on novel inputs.

Perturbation rate	Val. macro-F1
0.00	0.5106
0.25	0.4856
0.50	0.4673
0.75	0.4391
1.00	0.4150

Table 7.7: Effect of the base perturbation rate on validation macro-F1 (base size, eight epochs, single seed). The best cell on validation is in bold.

The synthetic multi-CoD sweep (Table 7.8) is best at ratio 0.3, a gain of 0.078 validation macro-F1 over ratio 0.

Synthetic multi-CoD ratio	Val. macro-F1
0	0.4673
0.15	0.5140
0.30	0.5453

Table 7.8: Effect of the synthetic multi-CoD ratio on validation macro-F1 (base size, eight epochs, single seed). The best cell is in bold.

The scheduler sweep (Table 7.9) is best with constant-with-warmup, 0.092 validation macro-F1 ahead of cosine.

LR schedule	Val. macro-F1
cosine	0.4673
linear	0.4642
constant-with-warmup	0.5595

Table 7.9: Effect of the learning-rate schedule on validation macro-F1 (base size, eight epochs, single seed). The best cell is in bold.

The input-features sweep (Table 7.10) leaves overall validation macro-F1 flat (a 0.0004 difference), but appending age and sex raises both the seen-string and unseen-string slices.

Input features	Val. macro-F1	Seen slice	Unseen slice
CoD	0.4673	0.749	0.345
CoD + age + sex	0.4669	0.905	0.437

Table 7.10: Effect of appending age and sex on validation macro-F1, with macro-F1 on the seen-string and unseen-string slices of the validation set (the validation records whose normalized cause string does, and does not, appear in training). Base size, eight epochs, single seed.

An exploratory probe also tested injecting masterlist code-description rows directly into the fine-tuning set, at 10 and 20 copies per label, rather than using them only for pretraining. It was measured only at the large size for 32 epochs on cause text alone, and injection did not improve validation macro-F1 (0.871 with injection off, against 0.858 and 0.861 at 10 and 20 rows per label), so it was not adopted. Because this probe was run off the base, eight-epoch ablation contract, it is reported here as a scouting result rather than a controlled ablation.

Masterlist pretraining is evaluated on transfer rather than in distribution, where it leaves accuracy essentially unchanged. Table 7.11 reports leave-one-source-out test macro-F1 with and without the pretraining stage (eight-epoch base runs), and pretraining improves three of the five fully held-out sources. This is the early base-configuration pretraining ablation, a different experiment from the deployment-recipe holdout models of Table 7.12, so the two tables’ macro-F1 values for the same source are not directly comparable.

Held-out source	No pretraining	With pretraining
Amsterdam	0.1070	0.1134
Belgium	0.2403	0.2642
Copenhagen	0.1994	0.1721
Ipswich	0.1870	0.2194
Madrid	0.1187	0.1218

Table 7.11: Test macro-F1 on each fully held-out source under leave-one-source-out training (base size, eight epochs), with and without masterlist pretraining. The stronger of the two settings is in bold per source.

7.4 Transfer to held-out sources

Each *codLLM-base holdout* model is trained on four sources with the deployment recipe and evaluated on the fifth, fully held-out source, on the cause-of-death string alone (this section answers RQ3). These are five separate models, not the deployment model, so they are named *codLLM-base holdout* rather than *codLLM-base-deploy*.

Table 7.12 reports exact-match accuracy from 0.220 to 0.639, Jaccard from 0.411 to 0.718, macro-F1 from 0.092 to 0.360, and micro-F1 from 0.438 to 0.730.

Held-out source	n	Exact-match acc	Jaccard	Macro-F1	Micro-F1
Amsterdam	575,973	0.299	0.465	0.104	0.485
Belgium	42,011	0.591	0.624	0.292	0.632
Copenhagen	501,855	0.220	0.411	0.095	0.438
Ipswich	38,927	0.639	0.718	0.360	0.730
Madrid	366,282	0.366	0.475	0.092	0.445

Table 7.12: Exact-match accuracy, Jaccard, macro-F1, and micro-F1 of the five *codLLM-base* leave-one-source-out holdout models, each on its fully held-out source.

Figure 7.4 reports the same held-out predictions after truncating codes to first-letter, three-character, and full ICD10h levels. The full six-level breakdown for every source is tabulated in Appendix H.

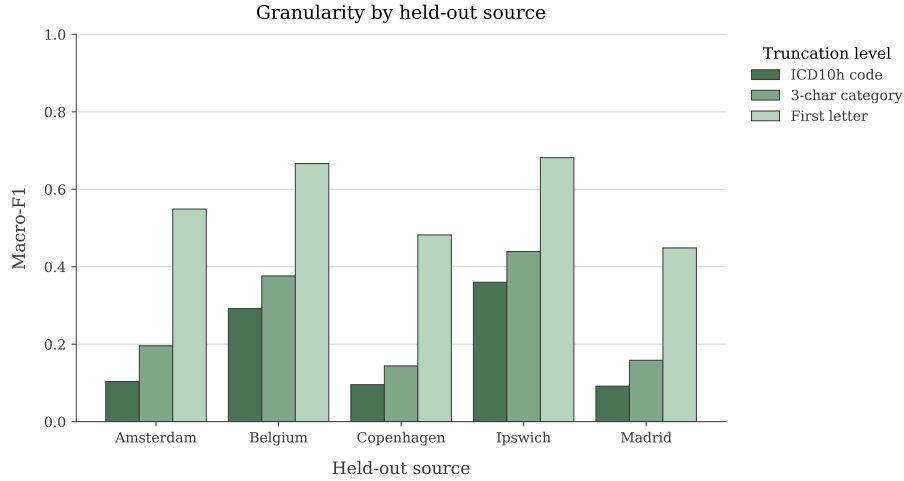


Figure 7.4: Macro-F1 of the five `codLLM-base` holdout models on each held-out source when predicted and true codes are truncated to the full ICD10h code, the three-character category, and the first letter. The full six-level breakdown for all five sources is given in Appendix H.

Figure 7.5 ranks each source’s held-out predictions by mean token entropy and retains the most confident first. At 80% coverage, retained accuracy is highest for Ipswich and Belgium. A leakage audit finds 12.6% of Belgium’s and 16.6% of Ipswich’s held-out records are exact (cause string, code) duplicates of training rows.

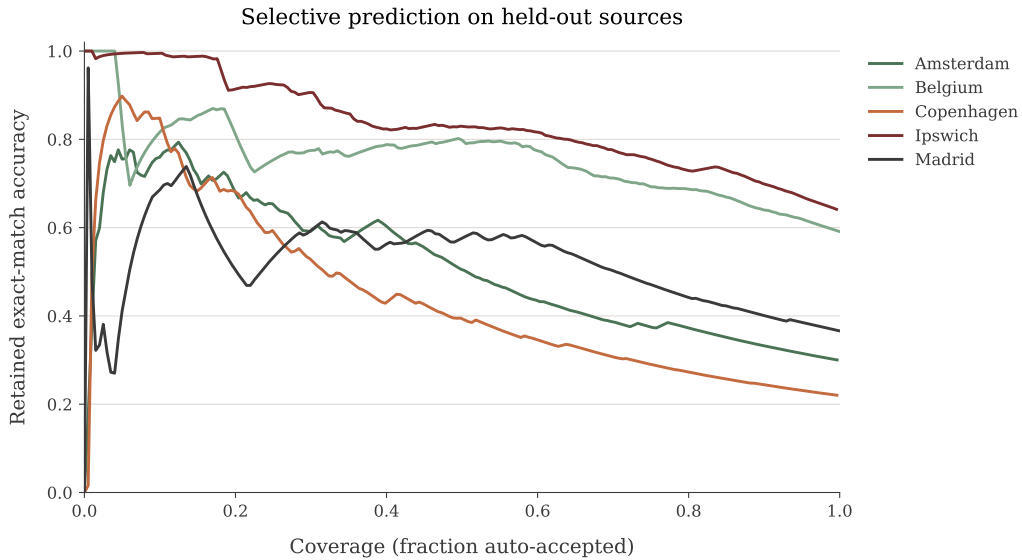


Figure 7.5: Retained exact-match accuracy against coverage on each leave-one-source-out `codLLM-base` holdout model, ranking held-out records by mean token entropy and retaining the most confident first. Belgium and Ipswich sit highest.

8 Discussion

8.1 Interpreting the results

Capacity saturates (model size). Additional capacity beyond the large model buys almost nothing under the size-sweep recipe, which is why the deployment recommendation centers on the base size rather than xl. Accuracy and micro-F1 sit near saturation across all sizes (0.977 to 0.988) as a consequence of the long-tailed label distribution, where the most frequent codes, which dominate the record count, are predicted almost perfectly, so a record-weighted metric is largely insensitive to capacity, whereas macro-F1, which weighs every code equally, is the metric that exposes the effect of size, indicating larger models to pick up on more nuance. The deployment recipe is where the base size earns its keep: at fixed base capacity it lifts macro-F1 from 0.723 to 0.749 and the unseen-slice macro-F1 from 0.589 to 0.625, recovering about 72% of the macro-F1 gap to the large model (0.759) at near-identical record-weighted accuracy (Table 7.1); the larger models were not retrained under this recipe, so this is a gain at fixed base size rather than a claim that the recipe beats them (subsection 8.4.2). The recipe therefore buys close to a full capacity step of rare-code coverage at the base size, which is the core of the deployment recommendation.

Coding specificity carries a deployment message. The specificity pattern is the same at every size and carries a clear message for deployment. At the chapter level the task is essentially solved (macro-F1 0.97 to 0.98), so a historical demographer who needs only the broad cause group can trust the model almost unconditionally. Specificity then costs accuracy in a front-loaded way. Most of the drop occurs between the chapter and the three-character category as the long tail of rare categories appears, while the further steps to the four-character ICD10 code and the full ICD10h code cost almost nothing more. Model size matters more the finer the target, so capacity buys fine-grained resolution rather than broad-category accuracy. Accuracy and micro-F1 are record-weighted, so they are dominated by the common high-frequency codes and track the near-saturated headline result, whereas macro-F1 weights every code equally and is the metric that exposes the rare-code difficulty as the target gets finer.

Supervision beats training-free baselines. Because the fine-tuned model is supervised on this dataset while the retrieval and agentic baselines are zero-shot, the reading is that task-specific supervision decisively beats strong training-free methods (RQ1), including a frontier agent equipped with tools. The margin is wide on every metric: on the

full test `codLLM-base-deploy` reaches 0.985 exact-match accuracy, 0.987 micro-F1, and 0.749 macro-F1 (Table 7.4), against 0.007 accuracy and 0.314 macro-F1 for the TF-IDF classifier, 0.000 and 0.041 for the multilingual-embedding retriever, and 0.039 and 0.0001 for the majority floor; on the matched 3,000-record subset it scores 0.983 accuracy and 0.897 macro-F1 against roughly 0.59 accuracy and 0.33 macro-F1 for the strongest agent (Table 7.5). None of the training-free methods learns to weight the extra age and sex fields, so for them the metadata acts as noise, most visibly on infant-death records whose age and sex do not match a maternal cause. For the supervised model the same fields work the other way: the input-features ablation (Table 7.10) leaves aggregate validation macro-F1 essentially unchanged (0.4673 against 0.4669) yet appending age and sex sharply lifts both the seen-string slice (0.749 to 0.905) and the unseen-string slice (0.345 to 0.437), which is why the deployment recipe conditions on metadata even though the pooled metric hides the gain. The comparison is also lopsided in cost. Each agentic prediction takes 25 to 30 seconds and \$0.08 to \$0.15 of API-equivalent usage (subsubsection 6.2.3), so coding the full test split once would cost roughly \$5,800 with Sonnet or \$11,600 with Opus, against effectively free inference for the fine-tuned model once trained. The agent is the strongest training-free baseline, but it is also the only one that carries a recurring per-record cost, and that cost scales linearly with the dataset, which makes it the least viable of the baselines for the full-archive coding the task actually requires.

Selective prediction works as intended. A deployment that auto-accepts the most-confident 80% of `codLLM-base-deploy` predictions and routes the rest to a historical demographer codes the accepted records at near-perfect accuracy while flagging only one record in five, so the answer to RQ2 is yes. A useful operating point exists at which the auto-accepted predictions are correct at near-perfect rates. This near-perfect auto-accept band holds uniformly across all four sizes (every model exceeds 0.996 retained accuracy by 90% coverage, with area under the risk-coverage curve below 0.0012, Figure 7.2), so the deferral safeguard does not depend on model capacity. The unseen-slice result is the safeguard for the memorization gap of the size sweep. The model is less accurate on genuinely novel causes, but its confidence falls in step, so those records are deferred to a historical demographer rather than coded confidently and wrongly. Because retained accuracy on the unseen slice plateaus near 0.94 rather than the full test’s near-perfect level, the confidence signal reflects genuine uncertainty rather than recognition of seen strings, even if absolute capability on unfamiliar text remains the ceiling. A more conservative deployment is worth exploring in future work. Rather than auto-accepting any prediction, the model could present a short ranked list of its most likely codes for each record, for example the top five, each with the model’s confidence, and never code autonomously. This narrows the expert coder’s choice while leaving the final decision to them, turning the same per-prediction uncertainty into a decision-support tool that keeps the human

fully in control, at the cost of the throughput that automation buys.

8.2 Historical Data Shift and Robustness

For most machine learning tasks, at least as they are taught, the data used to train a model is representative of the entire data pool. This is not the case for the model we have developed. Our training data comes in chunks, each representing a certain time period for a certain city, in a certain language. What matters for deployment is therefore representation: whether the training data covers the language, local source conventions, label distribution, and multi-cause structure of the archive a historical demographer wants to code. When it does not, the model faces source shift rather than ordinary test-set generalization. This creates several problems. First, new languages can always be introduced. Second, the class distribution of a new dataset need not resemble ours. Third, we have no way of characterizing what lies outside the data we hold. To mitigate this, several methods and strategies are introduced below. As stated in chapter 1 (RQ3), our main goal is to create a model that can be used on new datasets. In what follows we present the reasoning behind the methods described in section 6.2 and discuss how each addresses this inherent problem.

Historical language and local conventions. Language drifts over time, and cause-of-death wording is local as well as multilingual. New terms appear, old terms change form, spelling varies by scribe and period, and a city, institution, or coding project may use conventions that do not occur elsewhere in the corpus. This is a representation problem before it is a modeling problem. If the training data contains related wording, fine-tuning can learn the mapping from local expression to ICD10h code. If it does not, the model must rely on the linguistic structure already present in FLAN-T5, the ICD10h masterlist/reference strings, and whatever subword overlap the tokenizer can extract.

subsection 5.3.1 and Table 5.2 show that this is not only a model issue. Some of the difficulty is already present in the data: the same uninformative wording can be attached to different ill-defined or administrative codes, plain disease names can follow local label conventions, and historical spellings, mixed-language strings, or damaged fragments are not evenly represented across sources. This matters when reading the qualitative diagnostics in Table 7.2. The R99 and E13/E14 patterns are not just arbitrary output mistakes. They sit on boundaries where the corpus itself contains source-specific coding practice. In the normal random split, every source contributes to training and test, so the model often has examples of the relevant convention. In leave-one-source-out evaluation, that convention is removed for the target archive, and the same phenomenon becomes source shift.

The tokenizer is part of this failure mode. As described in section 3.2, FLAN-T5 uses

SentencePiece subwords, which prevents a historical spelling from becoming truly out of vocabulary. But rare archaic spellings, transcription variants, and local abbreviations are often split into longer and less semantically stable token sequences. That fragmentation can make rare strings harder to learn and can produce confident errors when an unfamiliar form resembles the wrong known pattern. The methods in this thesis address different parts of that risk. Character-level perturbation exposes the model to small spelling and transcription changes during training. Masterlist pretraining anchors the model in the ICD10h vocabulary and its English reference strings. Leave-one-source-out evaluation tests what remains when local representation is removed. Selective prediction is the deployment safeguard that routes low-confidence cases to review. This is the mechanism shown in Table 3.1: subword tokenization prevents true out-of-vocabulary inputs, but rare historical forms are represented by longer, more fragmented token sequences than familiar strings.

Leave-one-source-out as a lower-bound test. As described in section 6.2, we developed a method that excludes a given source from training and then measures how well the model performs on it. This should be read as a deliberately pessimistic stress test rather than as the normal deployment scenario. In ordinary use, we expect historical demographers to use `codLLM` when their language, area or spelling conventions are represented in the training data, or after they have added (or pretrained with) labeled material from their own archive. The holdout setting removes exactly that evidence. It asks a harder question: if a historical demographer brings a completely new source that is not represented in training, does the model still recover any useful coding signal at all?

On that reading, the holdout results are less a story of failure than of surprisingly strong transfer under an intentionally harsh condition. The model does not simply inherit its random-split performance: raw accuracy and macro-F1 fall when the entire target source is removed from training. But the important result is that performance does not collapse to zero. Even when deprived of source-specific examples, the model still maps a substantial share of records to plausible ICD10h labels: full-code exact-match accuracy runs from 0.22 to 0.64 across held-out sources and macro-F1 from 0.09 to 0.36 (Table 7.12), while the broad ICD chapter is still recovered for 0.61 to 0.85 of records even where the full code is wrong (Figure 7.4). The predictions therefore degrade toward coarser levels of the hierarchy rather than failing outright. The same gap appears between record-weighted and code-weighted metrics: held-out micro-F1 stays well above macro-F1 on every source (0.49 against 0.10 on Amsterdam and 0.73 against 0.36 on Ipswich at the full code, Appendix H), so the model carries over the frequent, general causes that dominate the record count, while the long tail of rare codes is what fails to transfer. The predictions remain useful enough to support triage and selective review.

Partial representation explains Belgium and Ipswich. The held-out-source results make this interpretation sharper because the held-out sources are not equally unseen. Belgium and Ipswich perform much better than Amsterdam, Copenhagen, and Madrid (exact-match accuracy 0.591 and 0.639 against 0.220 to 0.366, rising to 0.688 and 0.730 at 80% coverage, Table 7.12, Figure 7.5), and that difference is best understood as a gradient of representation. The Belgian holdout contains Flemish/Dutch material, which overlaps heavily with the much larger Dutch Amsterdam source that remains in training, so a large part of Belgium’s vocabulary is already represented even though Belgium itself is held out. This natural language overlap is the most direct explanation for why Belgium transfers better than the other held-out sources. Ipswich is English, the language of the masterlist reference strings and the language most strongly represented in the FLAN-T5 model. Crucially, the masterlist pretraining stage is itself in English, so that stage plausibly strengthens the model’s English coding ability and gives Ipswich an advantage no other held-out source receives. The leave-one-source-out pretraining ablation (Table 7.11) is consistent with this: masterlist pretraining nets a gain on three of the five held-out sources but regresses the Danish Copenhagen register (0.1994 to 0.1721), as expected if English-only pretraining helps the English and Dutch-overlapping sources while offering nothing to a non-English source it competes with for capacity. In both cases the held-out source is nominally new, but important parts of its language and coding vocabulary are already represented elsewhere in the corpus or the masterlist. The asymmetry is visible in the other direction: holding out Amsterdam still hurts, even though Belgium remains in training, because the Belgian source is much smaller and cannot represent the breadth of the Amsterdam register. Part of this edge, however, is direct overlap rather than generalization. A leakage audit (section 7.4) finds that 12.6% of Belgium’s and 16.6% of Ipswich’s held-out records are exact (cause string, code) duplicates of training rows. That overlap is record-weighted, driven by a small set of common shared strings rather than broad lexical coverage, so it is the same shared vocabulary that makes these sources well represented, but it also means their held-out numbers partly reflect recall of seen records rather than transfer to genuinely new text.

This matters for how the results should be used. Leave-one-source-out evaluation should not be interpreted as the accuracy a typical user should expect after training on data in their own language. It is closer to a lower-bound scenario for a user who has no local representation at all. The Belgium and Ipswich results suggest that representation is protective: when a new dataset is linguistically or lexically covered by existing sources or by the masterlist, transfer improves substantially. When it is not covered, the model should be treated as decision support rather than an autonomous coder.

Confidence turns transfer into triage. What makes the model useful in that lower-bound setting is not raw transfer alone, but that its confidence drops on unfamiliar inputs.

A training-free baseline such as the TF-IDF classifier exposes at most a decision margin that we did not develop into a deferral rule, so as evaluated it codes every novel record silently, including the ones it gets wrong, whereas `codLLM` flags its own uncertainty and defers the records it cannot handle to a human coder. How much that deferral actually helps is the real test, and the held-out selective results (Figure 7.5) give a qualified answer. The confidence ranking stays informative under the shift, lifting retained accuracy when the least-confident predictions are deferred, but it does not recover the near-perfect auto-accept band the model has in distribution: no held-out source sustains 0.95 retained accuracy beyond 18% coverage (Figure 7.5). On a genuinely unfamiliar source essentially no coverage reaches an accuracy a historical demographer could trust unchecked. The model therefore does not fail silently the way a training-free baseline would, but it is not safe to code an unrepresented source autonomously either, and its honest role there is decision-support, ranking which records a historical demographer should review first while local labeled data are collected. The answer to RQ3 is therefore a qualified yes: accuracy falls on a held-out source and confidence falls with it, so failures are deferred rather than coded silently wrong, but no operating point reaches an accuracy that would license unsupervised coding.

Upsampling. The training distribution is steeply long-tailed, so without intervention the rare codes contribute almost nothing to the gradient and the model collapses onto the common causes. Floor upsampling lifts every code below a target count up to that floor, and the floor sweep (section 7.3) found a floor of 200 best at validation macro-F1 0.5202 against 0.4673 at floor 50. The gain is concentrated in macro-F1, which weights every code equally, while exact-match accuracy and micro-F1 barely move, because those record-weighted metrics are dominated by the common codes that were already learned. Upsampling by exact duplication risks overfitting to the few rare-code records that exist, so resampled copies are perturbed in proportion to how hard their class was upsampled (subsubsection 6.2.4), trading memorization of the exact training string for broader coverage of its spelling variants. The base perturbation-rate sweep (Table 7.7) is best at rate 0 on clean validation and falls as the rate rises, but the deployment recipe deliberately keeps rate 0.5, paying that small clean-text cost as a hedge against the spelling and transcription noise of unseen strings. `codLLM-base-deploy` therefore adopts floor 200 as the setting that most improves rare-code recall at negligible cost to the common-code accuracy that dominates the dataset.

Synthetic Multi-CoD. Most records carry a single cause and only a minority carry two or three, so the model sees few genuine examples of how to emit several codes at once. Synthetic multi-CoD generation manufactures additional multi-CoD targets by combining single-cause rows. It samples a cardinality from the observed multi-CoD distribution, joins

distinct single-code examples, and unions their labels (subsubsection 6.2.4). This presents each disease outside its usual co-occurrence context, so the model learns to predict a code from its own text rather than from the company it usually keeps, which is what a record combining two unfamiliar causes demands. The synthesis sweep (section 7.3) found a ratio of 0.3 best, and `codLLM-base-deploy` adopts it. Because evaluation compares predictions as unordered sets and training shuffles multi-CoD label order, the benefit shows up as recovering the right set of causes rather than reproducing a canonical sequence.

8.3 Error Severity

Table G.1 and subsection 7.2.1 answer a question that the headline accuracy does not. When the model is wrong, how wrong is it? The hierarchy-aware results show that most of the remaining difficulty lies below the broad chapter level. Chapter-level macro-F1 is close to saturation, while performance drops at the three-character category and full ICD10h code levels. This means that many exact-code failures are not arbitrary jumps across the ICD10h tree, but errors of specificity. The model often preserves the broad disease area while failing to choose the correct category or full ICD10h code. For `codLLM-base-deploy`, chapter-level (first-letter) macro-F1 is 0.979, dropping to 0.784 at the three-character category and 0.749 at the full ICD10h code.

The qualitative diagnostics make this more concrete. R99 is the largest ambiguous residual category in the `codLLM-base-deploy` output, with 40 of 2,334 true R99 records missed (a 1.7% miss rate). In the masterlist, R99 covers ill-defined or unspecified mortality, including “Cause-of-death unclear”, “Stated unknown”, “No cause given”, and strings that are not causes of death. As shown in Table 5.2, even the literal string “unknown” is not attached to one universal code across sources. Errors involving R99 are therefore different from ordinary disease confusions. They may reflect a boundary between an unclear cause and a codable disease term, or between two source-specific conventions for the same uninformative wording, rather than a medical choice between two diagnoses. The one recurring R99 confusion pair, Z02 $\rightarrow$ R99 (5 instances, “unknown”), is exactly this kind of administrative-versus-ill-defined boundary rather than a disease misclassification.

Some category confusions appear less severe because the two categories are closely related in the masterlist. The E13 $\rightarrow$ E14 confusion (8 instances) falls inside diabetes. E13 is “Other specified diabetes mellitus”, while E14 is “Unspecified diabetes mellitus”. These are still errors for the task, but they are not equivalent to confusing unrelated disease groups.

Other errors are more severe because they either cross disease processes or drop part of a multi-cause target. The I38 $\rightarrow$ I70 confusion (4 instances) moves within the circulatory chapter but from endocarditis or valvular heart disease to atherosclerosis. The T59, T71 $\rightarrow$ T59 pattern (4 instances) is different again: the model retains one of the two toxic-

gas and asphyxiation causes but drops the second in a multi-cause record. These cases suggest that error severity is not one-dimensional. A wrong prediction can be close in the hierarchy, close in wording, close historically, or incomplete because one cause in a multi-cause record is missing.

8.4 Limitations

The following section examines in depth the main issues and limitations we encountered.

8.4.1 Dataset Quality

One caveat of training on labeled data is that the labels must be of high quality. The model is only as good as the data it is trained on. One example where this was sadly not the case for us is gender- and age-specific illnesses. Some causes, such as "childbirth", can be a cause-of-death for both a mother and a child, with different ICD10h codes. Here the metadata would prove indispensable. However, if the people who classified the data did not themselves code these cases differently, coding the strings alone without considering anything more, the model cannot learn the difference. Many such cases arise, where the model learns no meaningful distinction because the data itself does not contain one.

8.4.2 Limitations of the configuration search

The deployment configuration was assembled through one-factor-at-a-time ablations, where each component (upsampling floor, synthetic multi-CoD ratio, learning-rate schedule, perturbation rate, and input features) was varied while the rest were held at the reference contract. This is not a joint search over the full multi-dimensional hyperparameter grid, which would be computationally intractable at this dataset and model scale. The deployment configuration is therefore the best-known combination we found rather than a proven best possible setting. Factors that interact could in principle behave differently when tuned together than when tuned in isolation.

Two further caveats temper how strongly the ablations license each choice. First, the ablations are single-seed and short, eight fine-tuning epochs for most factors. Second, an eight-epoch advantage need not extrapolate identically to the 80-epoch deployment run. This caveat is most relevant for the learning-rate schedule, since constant-with-warmup lacks the decay that cosine applies over long training, so its eight-epoch margin may narrow at full budget. These caveats license the configuration choices as the strongest evidence available under the compute budget without overstating them.

The size sweep and `codLLM-base-deploy` also answer different questions. The size sweep isolates model capacity under one fixed recipe. `codLLM-base-deploy` changes three factors at base size only: upsampling floor, synthetic multi-CoD ratio, and learning-rate

schedule. Large and xl were not retrained under the deployment recipe, so we cannot claim that `codLLM-base-deploy` beats `codLLM-large` or `codLLM-xl` under a common recipe. The recommendation is instead to deploy at base size because the size sweep shows strong saturation above base, while the deployment recipe is the best-known base-size configuration available under the compute budget.

8.4.3 Output validity

Because the model generates each code as free text rather than selecting from a fixed list, its outputs are not valid by construction, and it can in principle emit a string that is not a real ICD10h code. In practice this almost never happens. Most test records carry a single cause and some carry two or three, so the model’s predictions across the test set total 84,553 codes for `codLLM-base` (84,552 for `codLLM-xl`), somewhat above the number of records. Of these, none were malformed in format and only 3 (`codLLM-base`) and 1 (`codLLM-xl`) were well-formed strings absent from the ICD10h masterlist, 0.004% and 0.001% of generated codes respectively. Constrained decoding, which restricts generation to valid codes and has been used for generative ICD coding of death certificates [27], would remove even these few cases. Given how rare invalid outputs already are, we did not adopt it, but it is a natural refinement for a deployment that requires a hard validity guarantee.

8.4.4 What the unseen-string slice measures

The unseen-string slice is the thesis’s main guard against memorization, but it is an imperfect proxy for genuine novelty and should be read as such. By construction it holds the test strings absent from training, and because the dataset is heavily duplicated (about 92% of test records share their exact string with a training record), the strings that happen to be unseen are overwhelmingly the rare ones: a common cause recurs often enough to land on both sides of the split, while a string seen only once or twice can fall entirely into the test set. “Unseen” is therefore strongly confounded with “rare”, so the lower accuracy on this slice reflects the long-tailed difficulty of uncommon causes as much as the novelty of the surface form. The confound also runs the other way: a string can be unseen as a whole yet be a trivial spelling or transcription variant of a very common training string, which the SentencePiece subwords (section 3.2) let the model handle almost as if it had seen it. The slice is thus best understood as a conservative but noisy lower bound on memorization-free performance, rather than a clean test of transfer to genuinely new vocabulary, the role the leave-one-source-out evaluation (section 7.4) is designed to fill.

8.4.5 Statistical scope of the reported intervals

The confidence intervals in chapter 7 are deliberately conservative in what they claim. They come from a cluster bootstrap that resamples distinct cause-of-death strings rather than individual records, which corrects for the same heavy duplication, but they still describe a single fixed model on a single dataset under one data seed. They are therefore a lower bound on the true uncertainty. They capture sampling and duplication variation but not variation across training seeds, data splits, or unseen sources. Macro-F1 is reported without an interval because a resampling interval over a long tail of singleton codes (501 of the codes observed in the test set appear once) reflects which singletons happen to be drawn rather than genuine model uncertainty. The honest reading is that a retrained model would vary more than these intervals suggest once seed and split variation are included, and the transfer results (section 7.4) are the clearest reminder that performance on a genuinely new source is not bounded by an in-distribution interval at all.

8.5 Societal Impact

The immediate value of a model like `codLLM` is practical. Historical cause-of-death coding is slow, specialized work. Each free-text entry must be read, interpreted in its historical and linguistic context, and mapped to a standardized disease category. A model that can do this at scale does not remove the need for expert judgment, but it changes where that judgment is spent. Instead of coding every record manually, a historical demographer can review uncertain cases, inspect systematic errors, and reserve manual effort for the records where it matters most. In that sense, the model is best understood as a workflow accelerator and triage tool, not as a replacement for domain expertise.

This also changes which sources become usable. Some registers may have been too large, too noisy, or too weakly documented to justify full manual coding. If a model can produce a first pass quickly, those sources become candidates for analysis, where a researcher can code the dataset, inspect aggregate patterns, and then decide whether the source is worth deeper manual attention. Even imperfect automatic coding can therefore expand the practical boundary of historical demographic research, especially when combined with selective prediction so that the least reliable records are flagged rather than silently accepted.

A further benefit is standardization. Human coding remains the reference standard, but it is not automatically transparent. Different coders may apply conventions differently, and unless records are double-coded or audited, the resulting variation can be hard to measure. This is not only hypothetical: the data examples in Table 5.2 show that similar wording and even uninformative labels can be attached to different codes across datasets. A shared model and documented coding pipeline would not remove the need for local

historical judgment, but it would make those cross-dataset disagreements easier to surface, audit, and correct. A trained model introduces its own errors, but those errors are at least reproducible and quantifiable. The same input receives the same output under the same model version, and the error profile can be reported by source, category, confidence, and severity. This does not make the model more authoritative than a human expert, but it makes its limitations easier to inspect and understand.

The societal risk follows from the same property. A standardized model can also standardize mistakes, especially for languages, regions, time periods, or rare causes that are underrepresented in training. The appropriate use is therefore not blind automation, but documented assistance: model version, training data, evaluation performance, and known failure modes should accompany any coded dataset produced with the system. Used in this way, automatic ICD10h coding can increase the scale and reproducibility of historical disease research while keeping interpretive responsibility with the researcher.

8.6 Systematic Bias

The subgroup results in subsection 7.2.2 do not indicate an impactful systematic bias by recorded sex. Male and female records differ by only 0.001 exact-match accuracy on the full test set (0.986 vs. 0.985), and by under one percentage point on the unseen-string slice (0.847 vs. 0.840). Given the size of the dataset and the much larger differences observed across sources, this is not a meaningful deployment concern in the present results. The missing-sex group cannot be interpreted in the same way. It contains only 358 full-test records and almost none on the unseen slice, so its high full-test accuracy mostly reflects repeated strings rather than robust generalization.

Age shows more variation, but not in a pattern that suggests a simple systematic disadvantage against one age group. Full-test accuracy is high across all bands, from 0.975 to 0.994, and the unseen-string slice is lowest for the 5–14 and 15–44 bands. This may reflect the cause-of-death mix rather than age itself. Infant and old-age records often concentrate around common, recurrent disease categories, while working-age deaths include a broader range of external, occupational, infectious, and less repeated causes. The model may therefore be less biased against an age group than less reliable on the kinds of causes that are more common in that age group.

The strongest systematic performance difference is by source. Copenhagen is near-perfect even on the unseen-string slice, while Madrid falls sharply from 0.975 on the full test to 0.601 on unseen strings. Belgium and Ipswich are lower on the full test (0.938 and 0.936) but have small unseen supports. The leave-one-source-out results add a second view of the same issue: as the gradient of representation discussed above shows, how well the training data covers a source, not demographics, drives its accuracy (Table 7.12). This is better understood as source and domain shift than as demographic bias. The sources differ

in language, period, coding conventions, spelling, size, and duplication. The implication is practical. Pooled test performance is not enough to license blind use on a new archive. The model should be evaluated per source, low-confidence or unfamiliar-source records should be routed to review, and users should expect materially better performance once their language or source type is represented in training.

There is also a label-distribution bias inherent in the task. Common causes dominate record-weighted metrics and are learned very well, while rare codes remain harder and are better reflected by macro-F1 and the error-severity analysis. This is not a demographic bias in the usual sense, but it matters for historical inference. If rare causes are systematically under-detected, downstream estimates of uncommon diseases may be biased even when overall accuracy is high. The upsampling, perturbation, and macro-F1 selection strategy are intended to reduce this imbalance, but they do not remove it. The safest interpretation is therefore that `codLLM` shows no large systematic bias by sex or age in the reported subgroup tests, while source shift and rare-code underrepresentation remain the main systematic risks.

9 Conclusion

This thesis asked whether a fine-tuned, task-specific language model can take over the bulk of manual ICD10h coding within a human-in-the-loop workflow of historical, multilingual causes of death, and whether the confidence of its predictions can make such a model safe to deploy. We fine-tuned FLAN-T5 at four sizes (`codLLM-small` through `codLLM-xl`) on roughly 1.5 million hand-coded records from five European registers, and evaluated it on a locked test split against classical, retrieval-based, and agentic baselines, under uncertainty-based selective prediction, and under transfer to an entirely held-out source.

RQ1: capability and comparison. On historical, multilingual ICD10h coding, the fine-tuned model outperforms every baseline on every metric. The deployment model `codLLM-base-deploy` reaches 0.749 macro-F1, 0.987 micro-F1, and 0.985 exact-match accuracy on the full test, against 0.314, 0.532, and 0.007 for the strongest training-free baseline, a TF-IDF linear classifier. On macro-F1, the metric that weights every code equally, that is more than twice the baseline, and the size sweep lifts macro-F1 further to 0.765 at xl. It also outpaces two agentic baselines built on a frontier LLM equipped with tools: on a matched 3,000-record subset `codLLM-base-deploy` reaches 0.983 accuracy and 0.984 micro-F1 while the agents reach only 0.560 to 0.601 accuracy and 0.596 to 0.633 micro-F1, and cost \$200 to \$492 to score even that small subset once against `codLLM`’s negligible inference cost. On the memorization-free unseen-string slice the deployment model still reaches 0.625 macro-F1 and 0.843 exact-match accuracy (Table 7.1), so its advantage reflects genuine generalization rather than recall of seen strings; the training-free baselines were scored on the full test only, so this is `codLLM`’s own seen-to-unseen gap rather than a head-to-head on the slice. Task-specific supervision on a modest amount of in-domain data therefore beats strong training-free methods, including a frontier agent equipped with tools, and at a small fraction of the per-record cost.

RQ2: uncertainty-informed selective prediction. A confidence signal read from the model’s own token-level outputs supports a usable deployment workflow. Auto-accepting the most-confident 80% of `codLLM-base-deploy` predictions codes the accepted records at near-perfect accuracy (above 0.999 at 80% coverage) while deferring only one record in five to a human. The signal reflects genuine uncertainty rather than string recognition: on the unseen-string slice, where the model is genuinely weaker (0.843 at full coverage), the same deferral raises retained accuracy to about 0.925 at 80% coverage, so the model defers novel causes instead of coding them confidently and wrongly. The answer to RQ2 is therefore yes, a practically useful operating point exists at which automated

coding is reliable and the genuinely ambiguous minority is routed to expert review.

RQ3: worst-case source holdout. The leave-one-source-out setting is the hardest evaluation in the thesis because it withholds the local language and source evidence that a real deployment would normally include. Under that lower-bound condition, the `codLLM-base` holdout models still recover useful coding signal rather than collapsing: they code between 0.22 and 0.64 of a held-out source’s records exactly right, with macro-F1 from 0.09 to 0.36, far below the 0.985 exact-match accuracy and 0.749 macro-F1 the same recipe reaches in distribution but well above collapse. The confidence ranking stays informative under the shift, deferring the least-confident fifth lifts retained accuracy by roughly five to ten points per source. What does not survive is the near-perfect auto-accept band the model has in distribution: no held-out source sustains 0.95 retained accuracy beyond 18% coverage, so none reaches a confident auto-accept band. The answer to RQ3 is therefore a qualified yes: accuracy falls on a held-out source and confidence falls with it, so failures are deferred rather than coded silently wrong, but no operating point reaches the accuracy that would license unsupervised coding. The role therefore changes: on an unrepresented register, `codLLM` is a decision-support and triage tool rather than an autonomous coder. The result is not that transfer fails, but that representation is the key condition, users should expect the model to work best once their language or source type is present in training, and to receive useful ranked assistance when it is not.

Contributions. This thesis contributes a fine-tuned sequence-to-sequence model for multilingual ICD10h coding of historical causes of death, with a controlled four-size capability study on a locked split; a comparison against classical, retrieval-based, and agentic baselines that quantifies how far task-specific supervision exceeds strong training-free methods; an uncertainty-based selective-prediction analysis that turns the model into a human-in-the-loop coder with a tunable accuracy-coverage trade-off; and an honest treatment of memorization, reporting results on an unseen-string slice and duplication-aware confidence intervals so that the headline numbers are not inflated by the dataset’s heavy duplication.

Recommendation. We recommend using the ready-to-use model described as `codLLM-base-deploy` throughout the thesis, which reaches 0.749 macro-F1, 0.987 micro-F1, and 0.985 exact-match accuracy on the full historical test set. The size recommendation is justified by the size sweep: accuracy and micro-F1 are saturated across all four sizes, the macro-F1 gain from large to xl is miniscule, and base has 300M parameters, making it runnable on most newer laptops. Paired with the selective-prediction threshold, our recommended usage allows the researcher to spend more time on CoD’s that need expert supervision.

Future work. The most direct next step is to add more datasets. The results consistently show that data representation is the central condition for a reliable model, so adding more languages, regions and periods should improve both ordinary test performance and robustness to new archives. A second direction is to make the system openly available through released code, trained models where permitted by the data-use agreements, and a free researcher-facing coding application built around the selective-prediction workflow. Such an application should expose the model’s uncertainty, apply an explicit confidence or coverage threshold, and let researchers focus their manual effort on low-confidence or otherwise hard records rather than treating the model as an autonomous authority. Third, the deployment recipe should be revisited with a full deployment (80 epoches) and broader hyperparameter search, for example Bayesian optimization over interacting settings such as upsampling, perturbation, synthetic multi-CoD generation, learning-rate schedule, and pretraining. Such results would still need to be interpreted cautiously, since the validation and test splits are not guaranteed to represent genuinely new sources. Finally, future work should explore stronger data synthesis methods, including translating masterlist entries or labeled examples into additional languages and training on those synthetic multilingual variants. This may improve transfer to languages and historical spellings that are only weakly represented in the current corpus.

10 Acknowledgments

We would like to thank our supervisors, Nicki Detlefsen, Tim Riswick, and Mads Perner, for their guidance throughout the project. Their feedback helped shape both the technical direction of the work and the historical framing of the thesis.

We are also grateful to Christian Vedel and Christian Møller Dahl for their work and for the discussions we had with them during the project. Their paper and our conversations helped clarify the practical needs of historical demographers and informed several of the methodological choices in this thesis.

This thesis depends on data that was assembled, curated, and shared by several research groups and archives. We thank Louise Villefrance Perner, Mads Perner, and the Copenhagen City Archives for the Copenhagen material; Angélique Janssens, Tim Riswick, and the Radboud Group for Historical Demography and Family History at Radboud University for the Amsterdam material; Eilidh Garrett and Alice Reid for the Ipswich material; Sven Vrielinck and Isabelle Devos of the Quetelet Center at Ghent University for the Belgium material; and Diego Ramiro Fariñas, Stanislao Mazzoni, Mélanie Bourguignon, Begoña Villuendas Hijosa, and Michel Oris for the Madrid material. We also thank the creators of ICD10h for making the classification system and masterlist available for research.

We especially thank Mads Perner and Tim Riswick for helping initiate the project and for contributing ideas, criticism, and encouragement during its exploratory phase. Mads Perner helped us understand the needs of historical demographers and how these should shape the model, while Tim Riswick helped establish contact with data providers and made access possible in ways we could not have achieved alone.

We would also like to thank everyone who has supported us throughout the project, including partners, family and friends.

Finally, we thank PandemiX and Lone Simonsen for providing a research environment in which the broader relevance of historical disease data was always present. The project was strengthened by being connected to a setting where past epidemics, disease patterns, and public-health history are studied not only as historical questions, but also as knowledge that can help interpret future outbreaks.

Appendix

A Declaration of Generative AI Usage

This appendix declares the generative-AI tools used in preparing this thesis, in keeping with DTU’s guidelines on the use of generative AI in written examination work. It concerns tools used to *produce* the thesis and its software, and is separate from the large language models evaluated as agentic baselines in chapter 6 and chapter 7, which are objects of study rather than authoring aids.

Tools and how we used them. We used ChatGPT Codex 5.5 and Claude. The research ideas and design are our own. We did sometimes use these tools as a vibe check to discuss the viability of an approach. In writing, they assisted with language editing, rephrasing, and the clarity of passages we had already drafted. In code, they helped with completion, debugging, and refactoring of the experimental and analysis code. They also helped with restructuring code into a more well-rounded codebase, like moving functions around and making a more clean codebase structure. Everything was reviewed before implementation.

Scope and responsibility. We did not use generative AI to fabricate, alter, or selectively report any data, result, figure, or table. All quantitative results derive from the experiments described in chapter 6 and chapter 7. The authors are solely responsible for the content of this thesis, including any errors.

Appendix

B.1 Data-use agreement

DTU Compute
Department of Applied Mathematics and Computer Science

DATA SHARING AGREEMENT

Elias D. Lunøe, Lucas B. Mortensen, Nicki S. Detlefsen
Technical University of Denmark

Project: Automated Coding of Historical Causes of Death using Fine-Tuned Large Language Models

1. The data provided for the project will only be used for the research project mentioned in the data request form.
2. The recipients will not share the data with any third-parties.
3. The recipients will implement reasonable technical and organizational measures to prevent re-identification and to minimize the risk of memorization or extraction of records from any trained model.
4. The recipients ensure that all parties providing data will have complete and full access to the model which is developed.
5. Data delivered by the provider(s) will consist of a selection of the appropriate data and metadata deemed necessary to carry out the research project.
 - a. **[required]** The selection will consist of the original, individualized strings of the causes of death (not including any personal information), the ICD10 and ICD10h codes for each cause of death for a period between 1860-1940 (or whichever range the dataset is).
 - b. **[preferred, not required]** Due to ICD10h complications, certain causes of death are dependent on the age and/or gender of the individual. Therefore, to achieve an optimal model, recipients (*optionally*) request access to age and gender for the individuals. Any age and sex/gender data provided will be used solely for model training, validation, and error estimation, and not for demographic or historical analyses.
 - c. **[optional]** For internal validation and estimation of systematic errors present in the model, recipients wish to receive other metadata such as date of death, etc. These are not necessary, and can be excluded if these are not wished to be shared.
6. If the results of the research project lead to a scientific publication, the project leaders will make agreements on authorship and acknowledgements prior to the submission of the manuscript.
7. If the results of the research project lead to a scientific publication, the project leaders will ensure adequate reference to the dataset in the following format: [to be entered by data provider]

DATA REQUEST FORM

The undersigned project leader(s) declare that they have read the attached data sharing agreement, and that they agree to comply with it.

Name	Date	Signature

On behalf of the project:

Name	Date	Signature
Lucas B. Mortensen		
Elias D. Lunøe		
Nicki S. Detlefsen		

B.2 GitHub Repository

<https://github.com/murrodroid/codllm>

B.3 Compute environment snapshot

The following environment snapshot was collected from a scheduled DTU HPC H100 job on 9 June 2026. It records the system and package versions visible to the training process at runtime.

Component	Value
Cluster node	n-62-12-88
Operating system	AlmaLinux 9.8 (Olive Jaguar)
Kernel	Linux 6.1.174-1.el9.elrepo.x86_64
CPU	2 × AMD EPYC 9354 32-Core Processor
Logical CPUs	64
GPU	NVIDIA H100 PCIe
GPU memory	81,559 MiB
NVIDIA driver	595.71.05
Driver-supported CUDA version	13.2
Loaded CUDA module in job profile	12.2
uv	0.10.8
Python	3.13.2
PyTorch	2.10.0+cu128
PyTorch CUDA build	12.8
Transformers	5.1.0
Accelerate	1.12.0

Table B.1: Representative DTU HPC H100 runtime environment used for the training jobs.

Agentic baseline prompt

The agentic baseline (subsubsection 6.2.3) classifies each record with an isolated large-language-model agent that has tool access to a shell, file search, the open internet, and the ICD10h masterlist. Both models evaluated, Claude Sonnet 4.6 and Claude Opus 4.8, used the identical system prompt reproduced below. At inference time the per-record cause-of-death string is appended beneath the prompt, together with the recorded age and sex in the metadata input setting. The prompt is shown verbatim. The 1700–1950 period it states is a broad contextual hint rather than the dataset’s actual 1853–1939 span (Table 5.1).

```
You are an expert ICD10h medical coder for historical cause-of-death (
  CoD) records. Your task is to classify ONE CoD string -- which may
  describe one or several causes -- into between one and three
  ICD10h codes.
```

Context

- ICD10h is a domain-specific extension of ICD10 designed for 19th-20th -century mortality records. It has ~14,000 codes.
- Code format: one uppercase letter, two digits, a period, three digits (e.g. ‘A16.200’ = pulmonary tuberculosis, ‘J18.900’ = unspecified pneumonia).
- Input language can be Dutch, French, Latin, Danish, German, Spanish, Italian, English, or mixed. Period: 1700-1950.
- Vocabulary is often archaic, abbreviated, or misspelled.

Multi-cause records

```
Historical death certificates frequently list more than one cause (e.g.
  "pneumonia, senility" or "phthisis with cardiac failure"). The
  expected output for such a record is multiple codes joined by
  commas, ordered by clinical primacy (most likely underlying cause
  first).
```

- About **10%** of all records carry **multiple codes** (up to three).
- The vast majority are single-code records.
- **Output a single code when only one is supported by the text** -- do not invent secondary codes to look thorough. Spurious extra codes will be scored as false positives.
- If the text genuinely describes two or three distinct causes, emit two or three codes.

Resources available to you

You have full agentic access; use whichever combination is most efficient.

- ****Bash**** -- shell commands, including 'grep'/'awk' against the masterlist or 'python -c' one-liners.
- ****WebSearch / WebFetch**** -- open internet. Use for translation, archaic-term lookup, etymology, historical-medicine references, and disambiguating obscure descriptions. ****This is what a 2026 historian working with an AI assistant would have access to****; use it freely when it would actually help.
- ****Read / Grep / Glob**** -- explore files in this repository.
- ****Masterlist file****: 'data/raw/ICD10h_Masterlist_2024.tsv'
 - Tab-separated, ~14,000 data rows + 1 header row.
 - Columns (in order): 'IDMasterlist', 'ICD10h', 'ICD10', 'icd10_2levelCATEGORY', 'ICD10_2levelCAUSE', 'ICD10h_DESCRIPTION', 'HistCat', 'DoNotUse', 'NotForUnderlying', 'GenderSpecific'.
 - Use 'grep -i' for case-insensitive keyword search; try multiple synonyms / root forms / anatomical sites / translated forms.
- ****Flag interpretation**** (guidance from the masterlist authors -- apply judgment, not hard filters):
 - 'DoNotUse=1': maintainers consider this code unsuitable for output.
 - 'NotForUnderlying=1': code is typically not used as an underlying cause-of-death.
 - 'GenderSpecific=1' male-only; '=2' female-only; '=0' no restriction.

Suggested strategy

1. If the term is unfamiliar or foreign, ****translate / expand**** it first (web search OK).
2. ****Search the masterlist**** for candidates by keyword. Try multiple queries -- synonyms, anatomical sites, translated forms, root words.
3. Decide whether the text describes ****one cause, two, or three****. Default to one unless the text clearly carries multiple.
4. For ambiguous historical terminology, choose the most clinically plausible code given the period context (pre-antibiotic, pre-vaccine, etc.).
5. ****Verify**** every emitted code exists in the masterlist before committing.

Output

Think and search as much as you need. End your response with ****exactly**

one JSON code block** -- and nothing after it. Order codes by your confidence, most likely first.

```
```json
{
 "codes": ["A16.200"],
 "reasoning": "<2-4 sentences explaining your decision and any
 disambiguation>",
 "confidence": "high|medium|low",
 "alternatives": ["A15.000", "B90.900"]
}
```
```

‘codes’ must be a non-empty array of 1-3 valid ICD10h codes. ‘alternatives’ are codes you considered but did not pick.

Manual-coding effort: source correspondence

This appendix reproduces, with permission, the email correspondence that establishes the manual-coding-effort figure cited in chapter 2. The Copenhagen burial records were hand-coded by Louise Villefrance Perner during her doctoral research. The resulting dataset is the Danish Historical Cause of Death Database [5]. Mads Perner (Roskilde University) supplied the recorded effort, roughly 351 hours, and granted permission both to reproduce this thread and to cite the dataset. The exchange below contains an early working estimate (about 51,000 unique strings at roughly 25 seconds each) drawn from a larger, raw working extract that still included the post-1911 records the correspondence later excludes. Recomputed on the archived dataset itself (10,551 distinct causes from 1861 to 1911, 2,554 of them coded to ICD10h), the same 351 hours give roughly two minutes per distinct cause, the figure cited in the main text. Email addresses are removed. The thread is otherwise unedited and in the original Danish.

Elias Dahl Lunøe to Mads Perner, 9 June 2026

Hej Mads, vi håber du har det godt. Vi er godt i gang med skriveprocessen af bachelorprojektet, men vi mangler et realistisk billede af, hvor lang tid den manuelle kodning faktisk tager, og du er nok den rette at spørge, eftersom Louise har kodet det københavnske datasæt. Vil du kunne give et nogenlunde konkret estimat på, hvor mange records man typisk kan nå at kode på en time eller en arbejdsdag, og hvor lang tid det samlet tog at kode det københavnske materiale? Til sidst: må vi citere dig med navn og titel i opgaven og vedlægge dit svar som bilag?

Mads Perner to Elias Dahl Lunøe, 9 June 2026

Hej begge! Louise har nogle timeregnskaber liggende fra sin ph.d.-tid, det fremgår at hun har brugt omkring 351 timer på den del af begravelsesprotokollerne, der er kodet. Hvordan passer det?

Elias Dahl Lunøe to Mads Perner, 9 June 2026

Hej Mads, tusind tak, det er en rigtig brugbar reference. Vi har kigget på det københavnske materiale, vi har fået: omkring 502.000 records, men kun cirka 51.000 unikke dødsårsags-formuleringer (resten er gentagelser). De 351 timer svarer altså til omkring 25 sekunder pr. unik formulering, hvilket passer

fint, hvis Louise har grupperet ens strenge og kodet hver gruppe én gang. Passer det med, hvordan hun arbejdede, og dækker de 351 timer nogenlunde det datasæt? Og må vi citere det i opgaven og lægge din mail som bilag? I så fald: hvordan vil Louise gerne krediteres?

Mads Perner to Elias Dahl Lunøe, June 2026

Hej begge. Du skal lave din beregning med udgangspunkt i vedhæftede ark, som er Louises “endelige” kodede datasæt. Hun havde kun begravelser frem til 1911, så du skal pille 1912–40 fra. Brug evt. bare sum-kolonnen i hendes ark, så datasættene ikke rodes sammen. Den tilgang du beskriver passer nogenlunde, ja. I skal citere denne arkivpakke når I bruger hendes koder: <https://arkivfinder.dk/kbharkiv/datasæt/8fa07314-e3be-11ef-aa8d-8ffffc26f48d>. Og I er velkomne til at vedlægge mailtråden som bilag.

Per-chapter accuracy

This appendix reports `codLLM-base` performance broken down by ICD10 chapter, as a secondary, descriptive check on whether accuracy tracks how often a chapter appears in the dataset. Table E.1 lists, for each chapter, the number of test records whose true code set includes a code in that chapter, together with exact-match accuracy and micro-F1 on those records. Both metrics are record-weighted and remain stable in small slices. Macro-F1 is omitted because several chapters hold only a few dozen records, where a per-class average is dominated by noise.

| Chapter | Name | n | Accuracy | Micro-F1 |
|---------|--------------------------|--------|----------|----------|
| I | Infectious and parasitic | 18,456 | 0.987 | 0.988 |
| X | Respiratory | 13,982 | 0.987 | 0.988 |
| XVIII | Symptoms and signs | 12,232 | 0.981 | 0.985 |
| IX | Circulatory | 11,535 | 0.985 | 0.987 |
| XVI | Perinatal conditions | 5,490 | 0.995 | 0.995 |
| II | Neoplasms | 4,956 | 0.968 | 0.971 |
| VI | Nervous system | 3,887 | 0.978 | 0.980 |
| XIV | Genitourinary | 2,829 | 0.971 | 0.976 |
| XI | Digestive | 2,710 | 0.960 | 0.967 |
| IV | Endocrine and metabolic | 1,589 | 0.979 | 0.983 |
| XX | External causes | 1,588 | 0.931 | 0.937 |
| XV | Pregnancy and childbirth | 791 | 0.946 | 0.949 |
| XIX | Injury and poisoning | 443 | 0.808 | 0.864 |
| V | Mental and behavioral | 422 | 0.946 | 0.962 |
| XIII | Musculoskeletal | 363 | 0.912 | 0.948 |
| III | Blood and immune | 272 | 0.934 | 0.952 |
| XII | Skin | 219 | 0.890 | 0.927 |
| XVII | Congenital malformations | 152 | 0.895 | 0.903 |
| XXI | Health-status factors | 64 | 0.875 | 0.909 |
| VIII | Ear | 62 | 0.871 | 0.909 |
| VII | Eye | 2 | 0.500 | 0.667 |

Table E.1: `codLLM-base` exact-match accuracy and micro-F1 by ICD10 chapter on the held-out test (reference strings excluded), sorted by chapter frequency. n counts test records whose true code set includes a code in that chapter, so the column sums above the test size (82,044 against 76,249 records) because multi-cause records span chapters. Accuracy and micro-F1 are record-weighted. Macro-F1 is omitted because several chapters are too thinly populated for a stable per-class average.

Accuracy tracks chapter frequency closely. The five most common chapters (infectious and parasitic, respiratory, symptoms and signs, circulatory, and perinatal conditions, each

above 5,000 records) are coded at 0.98 to 0.99, while the sparsest chapters fall to between 0.81 and 0.89 (injury and poisoning at 0.81, skin at 0.89, ear at 0.87). The Eye chapter holds only two test records and is reported for completeness. The pattern mirrors the long-tailed label distribution of the dataset. The model is most reliable on the common causes that dominate the record count and weakest on the rare chapters it has seen least during training.

Evaluation metric definitions

This appendix states in full the metrics summarized in section 3.6. For each record i , the predicted and true outputs are parsed into unordered ICD10h code sets P_i and G_i , with per-record counts

$$\text{tp}_i = |P_i \cap G_i|, \quad \text{fp}_i = |P_i \setminus G_i|, \quad \text{fn}_i = |G_i \setminus P_i|.$$

Exact-match accuracy. The fraction of records whose predicted set exactly equals the true set,

$$\text{ExactMatch} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[P_i = G_i].$$

Micro-F1. Pooling the per-record counts into $\text{TP} = \sum_i \text{tp}_i$, $\text{FP} = \sum_i \text{fp}_i$, and $\text{FN} = \sum_i \text{fn}_i$,

$$P_{\text{micro}} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad R_{\text{micro}} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad F1_{\text{micro}} = \frac{2 \text{TP}}{2 \text{TP} + \text{FP} + \text{FN}}.$$

Macro-F1. For each code c , with

$$\text{TP}_c = \sum_i \mathbf{1}[c \in G_i \wedge c \in P_i], \quad n_c^{\text{pred}} = \sum_i \mathbf{1}[c \in P_i], \quad n_c^{\text{true}} = \sum_i \mathbf{1}[c \in G_i],$$

the code-level scores are

$$P_c = \frac{\text{TP}_c}{n_c^{\text{pred}}}, \quad R_c = \frac{\text{TP}_c}{n_c^{\text{true}}}, \quad F1_c = \frac{2P_c R_c}{P_c + R_c},$$

with any zero-denominator ratio set to zero. Writing $\mathcal{C} = (\bigcup_i G_i) \cup (\bigcup_i P_i)$ for the set of codes that occur as a true or predicted label,

$$F1_{\text{macro}} = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} F1_c.$$

Jaccard similarity. The mean per-record set overlap,

$$\text{Jaccard} = \frac{1}{N} \sum_{i=1}^N \frac{|P_i \cap G_i|}{|P_i \cup G_i|}.$$

Specificity-level metrics

Figure 7.1 in the results chapter reports macro-F1 at three truncation levels of the ICD10h hierarchy for the four `codLLM` sizes and `codLLM-base-deploy`. The full six-level breakdown follows.

| Specificity level | Example | n | small | base | large | xl | base-deploy |
|--------------------------|---------|-------|-------|-------|-------|--------------|-------------|
| ICD10 chapter | I | 21 | 0.957 | 0.967 | 0.985 | 0.987 | 0.976 |
| First letter | A | 25 | 0.965 | 0.975 | 0.980 | 0.981 | 0.979 |
| First two characters | A1 | 200 | 0.823 | 0.836 | 0.874 | 0.887 | 0.872 |
| Three-character category | A16 | 670 | 0.710 | 0.767 | 0.801 | 0.807 | 0.784 |
| ICD10 code | A16.2 | 1,095 | 0.651 | 0.715 | 0.751 | 0.759 | 0.742 |
| ICD10h code | A16.200 | 1,465 | 0.657 | 0.723 | 0.759 | 0.765 | 0.749 |

Table G.1: Macro-F1 on the full test set when predicted and true codes are truncated to each level of the ICD10h hierarchy, for the `codLLM` size sweep and `codLLM-base-deploy`. The example column traces one code, `A16.200`, up the hierarchy, with the chapter identified by its Roman numeral rather than a code prefix. The two finest levels separate the four-character ICD10 code from the full ICD10h code, which appends the historical part. n is the number of distinct codes that occur in the test set, as a true label or a prediction, at that level, which are the classes macro-F1 averages over. They are a small fraction of the 14,088-code masterlist because the held-out test split contains only the codes that fall into it. Only macro-F1 is shown. The best size in each row is in bold. `codLLM-base-deploy` is the deployment model and sits between base and large at every level.

Held-out specificity-level metrics

Figure 7.4 in the results chapter reports the macro-F1 of the five **codLLM-base** leave-one-source-out holdout models at three truncation levels of the ICD10h hierarchy. The full six-level breakdown for all five held-out sources follows, in macro-F1 (Table H.1), exact-match accuracy (Table H.2), and micro-F1 (Table H.3). Each source is a separate model trained on the other four sources and evaluated on its fully held-out source, on the cause-of-death string alone. The number of evaluated records is 57,597 for Amsterdam, 4,201 for Belgium, 50,186 for Copenhagen, 3,893 for Ipswich, and 36,628 for Madrid.

| Specificity level | Example | Amsterdam | Belgium | Copenhagen | Ipswich | Madrid |
|-------------------|---------|-----------|---------|------------|---------|--------|
| ICD10 chapter | I | 0.619 | 0.714 | 0.481 | 0.727 | 0.590 |
| First letter | A | 0.549 | 0.666 | 0.482 | 0.682 | 0.449 |
| First two chars | A1 | 0.319 | 0.502 | 0.246 | 0.529 | 0.273 |
| 3-char category | A16 | 0.196 | 0.376 | 0.144 | 0.439 | 0.158 |
| ICD10 code | A16.2 | 0.124 | 0.319 | 0.094 | 0.389 | 0.093 |
| ICD10h code | A16.200 | 0.104 | 0.292 | 0.096 | 0.360 | 0.092 |

Table H.1: Macro-F1 of the five **codLLM-base** leave-one-source-out holdout models on each fully held-out source, averaged over the codes that occur at each level as a true label or a prediction. Macro-F1 weights every code equally and so is dominated by rare codes. It climbs steeply toward the coarser levels of the hierarchy.

| Specificity level | Example | Amsterdam | Belgium | Copenhagen | Ipswich | Madrid |
|-------------------|---------|-----------|---------|------------|---------|--------|
| ICD10 chapter | I | 0.776 | 0.865 | 0.611 | 0.842 | 0.786 |
| First letter | A | 0.774 | 0.855 | 0.608 | 0.832 | 0.786 |
| First two chars | A1 | 0.606 | 0.746 | 0.427 | 0.778 | 0.653 |
| 3-char category | A16 | 0.532 | 0.704 | 0.349 | 0.724 | 0.610 |
| ICD10 code | A16.2 | 0.385 | 0.659 | 0.279 | 0.710 | 0.429 |
| ICD10h code | A16.200 | 0.299 | 0.591 | 0.220 | 0.639 | 0.366 |

Table H.2: Exact-match (set) accuracy of the five **codLLM-base** leave-one-source-out holdout models on each fully held-out source, when predicted and true codes are truncated to each level of the ICD10h hierarchy. The example column traces one code, **A16.200**, up the hierarchy, with the chapter identified by its Roman numeral. Accuracy increases monotonically from the full code toward the chapter as the matching criterion coarsens.

| Specificity level | Example | Amsterdam | Belgium | Copenhagen | Ipswich | Madrid |
|-------------------|---------|-----------|---------|------------|---------|--------|
| ICD10 chapter | I | 0.844 | 0.896 | 0.721 | 0.917 | 0.830 |
| First letter | A | 0.840 | 0.881 | 0.714 | 0.905 | 0.827 |
| First two chars | A1 | 0.710 | 0.780 | 0.586 | 0.860 | 0.717 |
| 3-char category | A16 | 0.667 | 0.742 | 0.544 | 0.811 | 0.649 |
| ICD10 code | A16.2 | 0.573 | 0.696 | 0.479 | 0.792 | 0.495 |
| ICD10h code | A16.200 | 0.485 | 0.632 | 0.438 | 0.730 | 0.445 |

Table H.3: Micro-F1 of the five **codLLM-base** leave-one-source-out holdout models on each fully held-out source at each level of the ICD10h hierarchy. Micro-F1 pools true and predicted labels across all records and credits partial code overlap, so it sits above exact-match accuracy at every level.

Bibliography

- [1] Hyung Won Chung et al. “Scaling Instruction-Finetuned Language Models”. In: *Journal of Machine Learning Research* 25.70 (2024), pp. 1–53. URL: <https://jmlr.org/papers/v25/23-0870.html>.
- [2] World Health Organization. *International Statistical Classification of Diseases and Related Health Problems, 10th Revision (ICD-10)*. Fifth. World Health Organization. Geneva, Switzerland, 2016. URL: <https://icd.who.int/browse10/2016/en>.
- [3] Alice Reid et al. “From Deathbed to Database Using ICD10h: A Framework for Coding and Classifying Individual English Language Cause of Death Data Consistently Over Time, as Applied to Scotland 1855–1973”. In: *Social History of Medicine*, hkaf077 (2026). Advance access. DOI: [10.1093/shm/hkaf077](https://doi.org/10.1093/shm/hkaf077).
- [4] Alice Reid et al. *Historic cause of death coding and classification scheme for individual-level causes of death – Manual*. Apollo – University of Cambridge Repository. Method. 2024. DOI: [10.17863/CAM.109960](https://doi.org/10.17863/CAM.109960).
- [5] Louise Villefrance Perner. *The Danish Historical Cause of Death Database (DHCDD)*. Copenhagen City Archive (Københavns Stadsarkiv), Link-Lives project. Version 1.0. Causes of death from the Copenhagen Burial Register, 1861–1911, from the doctoral research of L. V. Perner. Licensed CC BY-SA. 2024. DOI: [10.71687/bvid.ksa.79](https://doi.org/10.71687/bvid.ksa.79). URL: <https://arkivfinder.dk/kbharkiv/datasaset/8fa07314-e3be-11ef-aa8d-8ffffc26f48d>.
- [6] Colin Raffel et al. “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer”. In: *Journal of Machine Learning Research* 21.140 (2020), pp. 1–67. URL: <https://jmlr.org/papers/v21/20-074.html>.
- [7] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. 2017. URL: <https://arxiv.org/abs/1706.03762>.
- [8] Taku Kudo and John Richardson. “SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing”. In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*. 2018, pp. 66–71. DOI: [10.18653/v1/D18-2012](https://doi.org/10.18653/v1/D18-2012).
- [9] Diederik P. Kingma and Jimmy Ba. “Adam: A Method for Stochastic Optimization”. In: *International Conference on Learning Representations (ICLR)*. 2015. URL: <https://arxiv.org/abs/1412.6980>.

- [10] Ilya Loshchilov and Frank Hutter. “SGDR: Stochastic Gradient Descent with Warm Restarts”. In: *International Conference on Learning Representations (ICLR)*. 2017. URL: <https://arxiv.org/abs/1608.03983>.
- [11] Priya Goyal et al. “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour”. In: *arXiv preprint arXiv:1706.02677* (2017). URL: <https://arxiv.org/abs/1706.02677>.
- [12] Chris M. Bishop. “Training with Noise is Equivalent to Tikhonov Regularization”. In: *Neural Computation* 7.1 (1995), pp. 108–116. DOI: [10.1162/neco.1995.7.1.108](https://doi.org/10.1162/neco.1995.7.1.108).
- [13] Yonatan Belinkov and Yonatan Bisk. “Synthetic and Natural Noise Both Break Neural Machine Translation”. In: *International Conference on Learning Representations (ICLR)*. 2018. URL: <https://arxiv.org/abs/1711.02173>.
- [14] Jason Wei and Kai Zou. “EDA: Easy Data Augmentation Techniques for Boosting Performance on Text Classification Tasks”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*. 2019, pp. 6382–6388. URL: <https://aclanthology.org/D19-1670/>.
- [15] Marina Sokolova and Guy Lapalme. “A Systematic Analysis of Performance Measures for Classification Tasks”. In: *Information Processing & Management* 45.4 (2009), pp. 427–437. DOI: [10.1016/j.ipm.2009.03.002](https://doi.org/10.1016/j.ipm.2009.03.002).
- [16] L. V. Ludvigsen et al. “Automated Coding of Historical Danish Cause of Death Data Using String Similarity”. In: *Digital Humanities in the Nordic and Baltic Countries Publications* 5.1 (2023), pp. 203–221. DOI: [10.5617/dhnbpub.10662](https://doi.org/10.5617/dhnbpub.10662).
- [17] Bárbara A. Revuelta-Eugercios, Helene Castenbrandt, and Anne Løkke. “Older Rationales and Other Challenges in Handling Causes of Death in Historical Individual-Level Databases: The Case of Copenhagen, 1880–1881”. In: *Social History of Medicine* 35.4 (2022), pp. 1116–1139. DOI: [10.1093/shm/hkab037](https://doi.org/10.1093/shm/hkab037).
- [18] Bjørn Pedersen et al. “Coding Historical Causes of Death Data with Large Language Models”. In: *Bridging the Gap Between AI and Reality (AISoLA 2024)*. Lecture Notes in Computer Science. arXiv:2405.07560. 2024. DOI: [10.1007/978-3-031-73741-1_3](https://doi.org/10.1007/978-3-031-73741-1_3).
- [19] Christian Møller Dahl, Torben Johansen, and Christian Vedel. “Breaking the HISCO Barrier: Automatic Occupational Standardization with OccCANINE”. In: *arXiv preprint arXiv:2402.13604* (2024). URL: <https://arxiv.org/abs/2402.13604>.
- [20] James Mullenbach et al. “Explainable Prediction of Medical Codes from Clinical Text”. In: *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*. 2018, pp. 1101–1111. DOI: [10.18653/v1/N18-1100](https://doi.org/10.18653/v1/N18-1100).

- [21] Thanh Vu, Dat Quoc Nguyen, and Anthony Nguyen. “A Label Attention Model for ICD Coding from Clinical Text”. In: *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI)*. 2020, pp. 3335–3341. DOI: [10.24963/ijcai.2020/461](https://doi.org/10.24963/ijcai.2020/461).
- [22] Chao-Wei Huang, Shang-Chi Tsai, and Yun-Nung Chen. “PLM-ICD: Automatic ICD Coding with Pretrained Language Models”. In: *Proceedings of the 4th Clinical Natural Language Processing Workshop*. 2022, pp. 10–20. DOI: [10.18653/v1/2022.clinicalnlp-1.2](https://doi.org/10.18653/v1/2022.clinicalnlp-1.2).
- [23] Alistair E. W. Johnson et al. “MIMIC-III, a Freely Accessible Critical Care Database”. In: *Scientific Data* 3 (2016), p. 160035. DOI: [10.1038/sdata.2016.35](https://doi.org/10.1038/sdata.2016.35).
- [24] Joakim Edin et al. “Automated Medical Coding on MIMIC-III and MIMIC-IV: A Critical Review and Replicability Study”. In: *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*. 2023, pp. 2572–2582. DOI: [10.1145/3539618.3591918](https://doi.org/10.1145/3539618.3591918).
- [25] Daniel Simig et al. “Open Vocabulary Extreme Classification Using Generative Models”. In: *Findings of the Association for Computational Linguistics: ACL 2022*. 2022, pp. 1561–1583. DOI: [10.18653/v1/2022.findings-acl.123](https://doi.org/10.18653/v1/2022.findings-acl.123).
- [26] Zhichao Yang et al. “Multi-label Few-shot ICD Coding as Autoregressive Generation with Prompt”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 37. 4. 2023, pp. 5366–5374. DOI: [10.1609/aaai.v37i4.25668](https://doi.org/10.1609/aaai.v37i4.25668).
- [27] Isabel Coutinho et al. “ICD Coding of Death Certificates with Generative Language Models”. In: *PLOS Digital Health* (2026). DOI: [10.1371/journal.pdig.0001245](https://doi.org/10.1371/journal.pdig.0001245).
- [28] Sophie Henning et al. “A Survey of Methods for Addressing Class Imbalance in Deep-Learning Based Natural Language Processing”. In: *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*. 2023, pp. 523–540. DOI: [10.18653/v1/2023.eacl-main.38](https://doi.org/10.18653/v1/2023.eacl-main.38).
- [29] Yonatan Geifman and Ran El-Yaniv. “Selective Classification for Deep Neural Networks”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. 2017, pp. 4878–4887. URL: <https://arxiv.org/abs/1705.08500>.
- [30] Dan Hendrycks and Kevin Gimpel. “A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks”. In: *International Conference on Learning Representations (ICLR)*. 2017. URL: <https://arxiv.org/abs/1610.02136>.
- [31] Chuan Guo et al. “On Calibration of Modern Neural Networks”. In: *Proceedings of the 34th International Conference on Machine Learning (ICML)*. Vol. 70. 2017, pp. 1321–1330. URL: <https://proceedings.mlr.press/v70/guo17a.html>.

- [32] Andrey Malinin and Mark Gales. “Uncertainty Estimation in Autoregressive Structured Prediction”. In: *International Conference on Learning Representations (ICLR)*. 2021. URL: <https://openreview.net/forum?id=jN5y-zb5Q7m>.
- [33] A. Janssens et al. *Amsterdam Cause-of-death Database, 1854–1940*. Version 1.0, unpublished dataset, Radboud Group for Historical Demography and Family History, Radboud University, The Netherlands. Provided for this thesis by T. Riswick. Acknowledgment wording specified by the data provider. 2023.
- [34] Diego Ramiro Fariñas et al. *Madrid Database of Causes of Death, 1905–1927*. Individual-level cause-of-death database for the city of Madrid, compiled over two decades and completed in 2025. Described in Ramiro Fariñas et al., “Individual-level data on causes of death in Madrid from 1905 to 1927” (working paper). 2025.
- [35] Sven Vrielinck and Isabelle Devos. *Causes of Death from the S.O.S. Antwerp Mortality Database, 1920–1930*. Individual-level death records (sex, age, written cause of death, and ICD-10h codes) extracted from the S.O.S. Antwerp mortality database, obtained from the Quetelet Center at Ghent University (project S.O.S. Antwerp). 2026.
- [36] Eilidh M. Garrett and Alice M. Reid. *Individual level cause of death strings with ICD10h codes, Ipswich 1871–1901*. Enhanced version of: P. Razzell, R. Davies and E. M. Garrett, *Sociological Study of Fertility and Mortality in Ipswich, 1872–1910* [computer file]. Colchester, Essex: UK Data Archive [distributor], August 2007. SN: 5413. 2025. DOI: [10.5255/UKDA-SN-5413-1](https://doi.org/10.5255/UKDA-SN-5413-1). URL: <http://dx.doi.org/10.5255/UKDA-SN-5413-1>.
- [37] Alice Reid et al. *Historic Cause of Death Coding and Classification Scheme for Individual-Level Causes of Death – Codes*. Dataset. 2024. DOI: [10.17863/CAM.109961](https://doi.org/10.17863/CAM.109961). URL: <https://doi.org/10.17863/CAM.109961> (visited on 06/09/2026).
- [38] Thomas Wolf et al. “Transformers: State-of-the-Art Natural Language Processing”. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Online: Association for Computational Linguistics, Oct. 2020, pp. 38–45. URL: <https://aclanthology.org/2020.emnlp-demos.6/>.
- [39] Lukas Biewald. *Experiment Tracking with Weights and Biases*. Software available from wandb.com. 2020. URL: <https://www.wandb.com/>.
- [40] Astral. *uv: An extremely fast Python package and project manager*. Version 0.9.21 (0dc9556ad 2025-12-30). 2026. URL: <https://github.com/astral-sh/uv>.

- [41] Suchin Gururangan et al. “Don’t Stop Pretraining: Adapt Language Models to Domains and Tasks”. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2020, pp. 8342–8360. DOI: [10.18653/v1/2020.acl-main.740](https://doi.org/10.18653/v1/2020.acl-main.740).
- [42] Jason Phang, Thibault Févry, and Samuel R. Bowman. “Sentence Encoders on STILTs: Supplementary Training on Intermediate Labeled-data Tasks”. In: arXiv:1811.01088. 2018. URL: <https://arxiv.org/abs/1811.01088>.